Smart Parking System

An IoT Architecture for Occupancy Monitoring, Allocation and Short-Horizon Forecasting

Akil · 25BCE2777
Jayaprakash V · 25BCE2766
Institution: VIT Vellore (vit.ac.in)
Course: Embedded System Architecture
Faculty / guide: Vivek Rajpoot
Updated project report · 31 October 2026

Abstract

Parking guidance needs current availability, vehicle records, and a clear rule for assigning spaces. This report describes a five-space prototype built around an ESP32, RFID access, a FastAPI service, SQLite, and a dashboard. A space becomes occupied after the server confirms an RFID entry and becomes free after a confirmed exit. Forecasting runs on the host computer in the current design. The synthetic dataset contains 4,032 five-minute observations and 27 features. LSTM, GRU, and Transformer regressors share a 12-step lookback, a one-step horizon, a chronological 70/15/15 split, and training-only scaling. On the synthetic test partition, GRU records MAE 0.098055 and RMSE 0.117862; persistence records 0.120875 and 0.169173, respectively. GA, PSO, ACO, and NSGA-II were evaluated with first-available and exhaustive references across on 120 SUMO-observed states, 3 repeats, and 2,376 bays. NSGA-II mean cost was 0.176095, with mean optimality gap 0.001936; exhaustive search is the exact one-slot reference. SUMO results do not represent field performance. Software checks cover API lifecycle, failures, allocation, and inference. Controller firmware was compiled, flashed, and hash-verified. In two post-rewire tabletop trials, the reader detected both scans and the API accepted both transactions (HTTP 200). Entry and exit response times were 1,431 ms and 1,220 ms. Server status showed S1 occupied and then free. Physical LED output, servo travel, current draw, and Wi-Fi recovery were not measured. The prototype and its synthetic data do not establish independent real-lot performance in the field. Larger trials and measured occupancy data are needed before deployment performance can be assessed independently.

Keywords: smart parking, Internet of Things, RFID, allocation optimization, occupancy forecasting, LSTM, GRU, Transformer

1. Introduction

Finding a parking space involves several linked decisions. Drivers need current availability, operators need a dependable arrival and departure record, and the system needs a rule for choosing among free spaces. Recent reviews identify data quality, forecast horizon, sensing method, and transfer between sites as recurring evaluation concerns [1], [2].

The design assigns local input/output tasks to an ESP32 and sends parking operations to a host computer. The controller reads an RFID tag and calls the vehicle-lifecycle API. FastAPI manages session records, slot allocation, forecasts, and dashboard data, while TensorFlow/Keras runs the forecasting models on the host. The ESP32 does not run neural inference.

This work contributes four connected pieces: an idempotent vehicle API with a shared allocation objective; a privacy-safe synthetic time series for comparing neural forecasts; a GA, PSO, ACO, and NSGA-II benchmark replayed on parking states observed during an executed SUMO/TraCI run, with first-available and exact references; and controller firmware that keeps the gate closed unless the server confirms an allocation. In the post-rewire tabletop trial, both RFID scans led to successful entry and exit transactions. That evidence shows the workflow operating, but the two trials are too few to characterize reliability. Physical outputs that were not directly observed are reported separately.

The Arduino framework gives the ESP32 a straightforward way to poll the RFID reader and control local outputs. Training and running the neural regressors requires more memory and processing capacity, so those tasks remain on the host computer. The split keeps device behavior bounded and leaves forecasting with the server.

2. Literature Survey

Recent parking studies address occupancy forecasts, networked access, and space allocation. Table 1 summarizes eight studies published from 2023 to 2026 and relates their methods to this project. Together, they motivate two focused evaluations here: an integrated five-space physical prototype and a reproducible allocation comparison across the 2,376 modeled garage bays, alongside prediction experiments using synthetic time-series data.

Table 1. Related studies and their relevance to this project

Study	Venue	Method / reported contribution	Relevance and limitation
Channamallu et al. [1]	2023 · Transportation Research Procedia	Reviews 108 studies on occupancy prediction, including statistical, recurrent and hybrid approaches.	Synthesizes modeling families but does not validate this project's five-slot integration or synthetic test set.
Bilotta et al. [2]	2023 · IEEE Access	Uses convolutional bidirectional LSTMs and feature-relevance analysis for free-slot forecasting from real garages.	Provides a real-data comparator; its garage observations do not establish generalization of the synthetic project data.
Zhang et al. [3]	2024 · ISPRS International Journal of Geo-Information	Compares TCN and LSTM and proposes MAT-LSTM for parking-space availability.	Uses real and synthetic datasets; the present evaluation uses its own synthetic generator and does not claim cross-site validation.
Enríquez et al. [4]	2024 · Applied System Innovation	Evaluates parking-flow prediction and fog/cloud response through a simulated deployment setup.	Separates model and network timing; this report likewise distinguishes server inference from physical controller measurements.
Kuang et al. [6]	2025 · Advanced Engineering Informatics	Meta-learning model fuses spatial relationships, temporal history and weather across 58 parking lots.	Addresses regional heterogeneity; this five-slot implementation has no cross-lot graph or measured weather data.
Kaltenbrunner et al. [7]	2025 · EPJ Data Science	Fits an explainable aggregate occupancy model to eight real park-and-ride facilities.	Shows a compact real-data alternative to deep models; the current neural comparison is limited to synthetic data.
Vakrinou et al. [9]	2026 · International Journal of Transportation Science and Technology	Compares LSTM-attention and Transformer models for short- and long-term prediction at multiple Athens facilities.	Finds horizon-dependent model strengths; this project reports only a one-step five-minute target.
Ostadi et al. [10]	2026 · Datenbank-Spektrum	Evaluates horizon-aware blended forecasting with sensor data from two urban parking facilities.	Highlights that one predictor may not lead at every horizon; this project compares models at one shared horizon.

Reported forecasting results depend on the horizon, input features, and whether the data comes from an operating facility [2]–[7], [9], [10]. Other IoT studies combine classification or prediction with optimization, but use different targets and objectives [5], [8]. This report addresses a narrower engineering question: can the models and allocation policies be compared on shared inputs while keeping software results distinct from the small hardware trial? The recurrent and self-attention methods used here build on [11]–[13].

The reviewed work ranges from city-scale forecasts to specific allocation formulations. This project pairs an inspectable five-space physical prototype with a six-level, 2,376-bay simulated garage. The controlled allocation comparison evaluates each heuristic and NSGA-II against exhaustive enumeration over all 2,376 modeled candidates. The physical prototype remains five spaces; the larger garage and its candidate objectives are simulated and do not validate real facility demand or traffic.

3. Problem Statement and Objectives

The system must maintain a consistent record of the five configured spaces, associate each entry and exit with an idempotent operation, and assign only a free space. It exposes current status and a short-horizon forecast through the API and dashboard. A space counts as occupied while its RFID vehicle session is active; the forecast is a regression estimate based on aggregated server history.

Table 2. Objectives, completion status, and available evidence

Objective	Status and evidence
Identify configured-slot occupancy	Implemented: server-confirmed RFID entry and exit update slot occupancy transactionally in SQLite.
RFID entry/exit	Implemented end-to-end; the latest tabletop trial recorded 2/2 tag reads and 2/2 HTTP 200 entry/exit transactions. Two trials do not establish reliability.
Compare GA, PSO, ACO, and NSGA-II allocation	Complete: 120 SUMO-observed allocation states × 3 optimizer repeats, 2,376 candidate bays, common objective, first-available and exhaustive references.
Forecast next-interval occupancy	Complete on deterministic synthetic data using LSTM, GRU, Transformer and persistence baseline.
Dashboard visibility	Dashboard and APIs are implemented; an actual browser capture is included. The physical RFID rig has five sensor spaces; the current dashboard also tracks the 2,376-bay garage emulation and displays live optimizer comparisons.
Low-cost hardware integration	Photographs document the tabletop ESP32/RC522/breadboard/LED assembly. Serial logs document two RFID transactions; visual LED output and physical servo motion remain unverified.

4. System Architecture

The backend coordinates each parking transaction. It writes the RFID session and slot assignment together, and the controller opens the demonstration gate only after it confirms the committed result. SQLite stores the authoritative state. The dashboard reads that same state through backend routes rather than querying the controller directly.

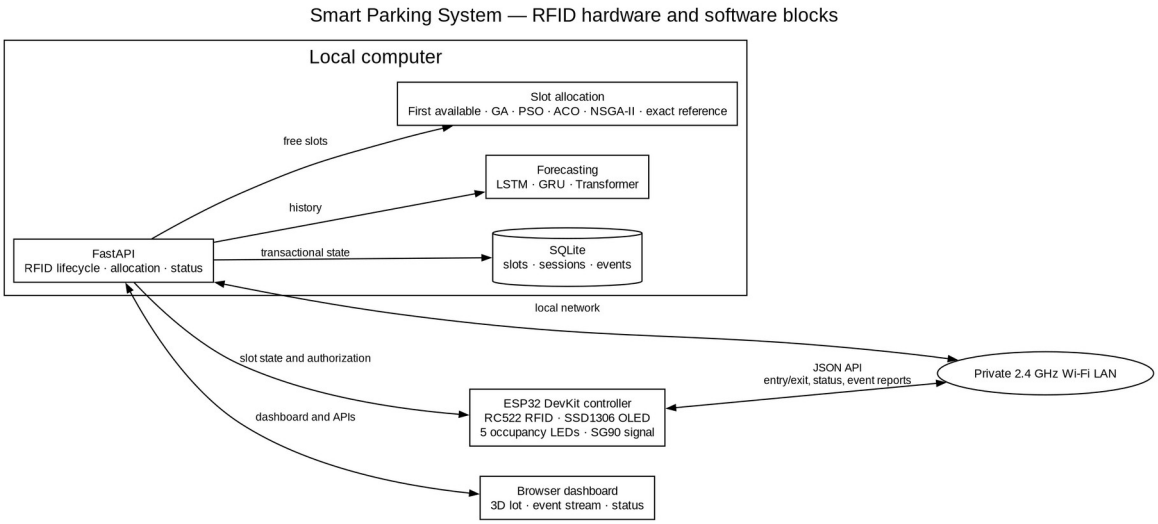


Figure 1(a). RFID transaction data flow, allocation, forecasting and persistence run on the server.

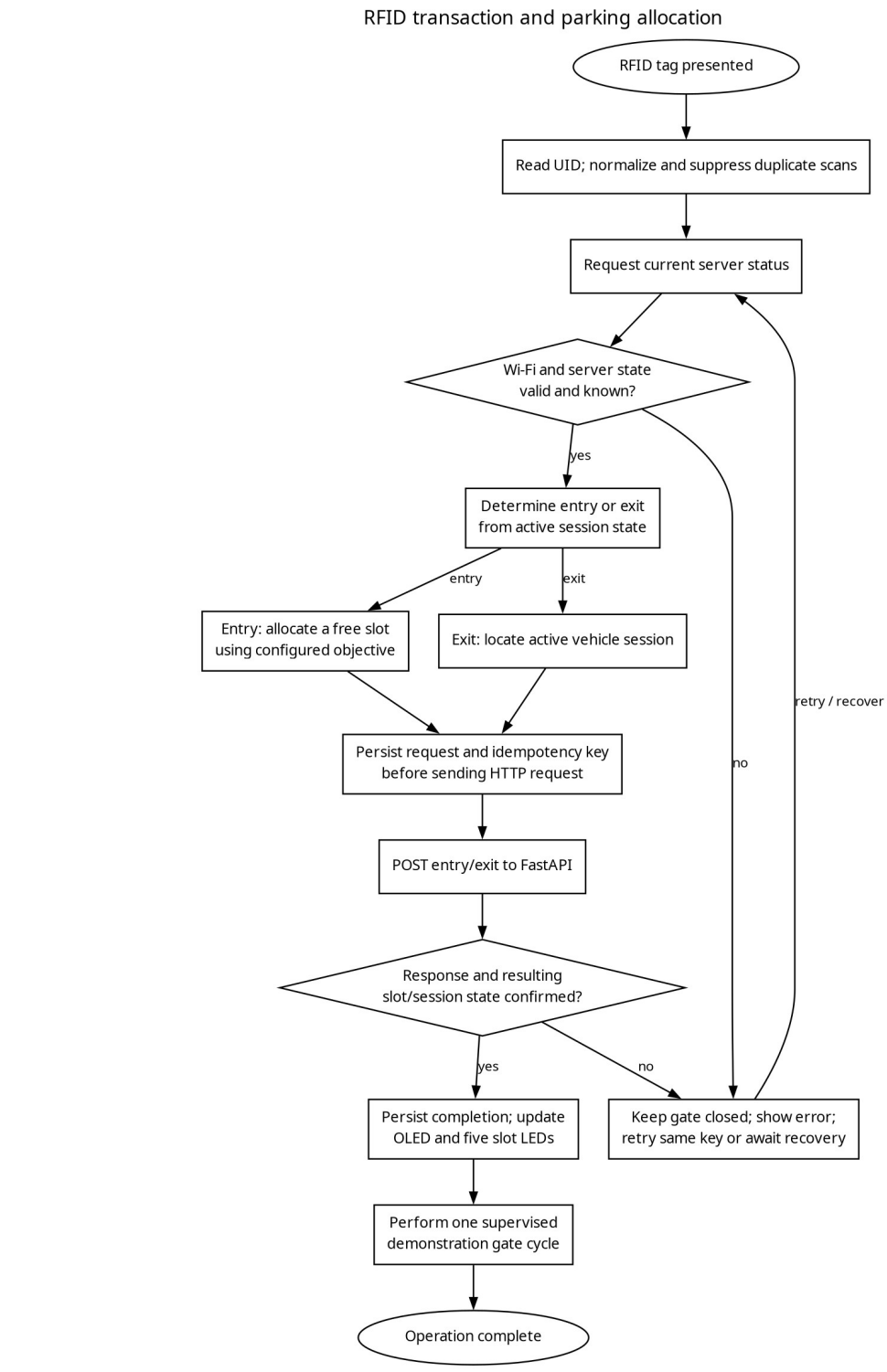


Figure 1(b). RFID transaction and allocation algorithm, including fail-closed error handling.

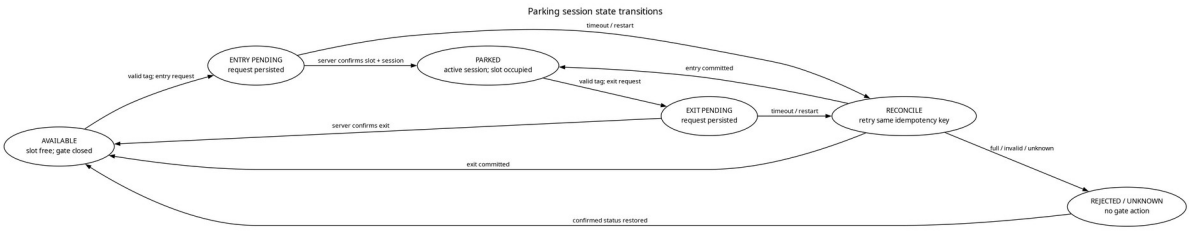


Figure 1(c). Vehicle-session state diagram; timeout recovery reuses the persisted idempotency key.

Figures 1(a)–1(c) show the embedded hardware and software data flow, RFID request algorithm, and vehicle-session state transitions. Controller flow: RC522 UID → GET /api/parking/status → POST /api/vehicle/entry

or /api/vehicle/exit with a persisted idempotency key → response and subsequent status validation → OLED/LED update and one gate cycle. A timeout preserves the pending key and leaves the gate closed until confirmation.

Forecast flow uses the same PreparedDataset contract for each model. The runtime predictor loads compatible, hash-checked model artifacts and uses a deterministic fallback when unavailable. Allocation uses configured weights and excludes occupied candidates. The optimizer benchmark captures the actual allocation request at each SUMO/TraCI parking admission, including all 2,376 candidate bays, their live occupancy, zone congestion, configured distance values, and identity-based fairness values. Each algorithm is replayed on the same captured request, and exhaustive search computes the exact single-slot reference for that observed state. Full candidate snapshots are preserved in the submission evidence.

The backend exposes separate routes for vehicle entry/exit, parking status, allocation benchmarks, forecasts, and dashboard data. The database is the authoritative state shared by those services. A slot is occupied while its RFID vehicle session is active and becomes free after a confirmed exit.

5. Hardware Design

The tabletop prototype uses an ESP32 DevKit, an RC522 reader, a solderless breadboard, indicator LEDs, and jumper wires. The firmware assigns RC522 SS to GPIO 5 and RST to GPIO 4; SPI uses GPIO 18/19/23, the OLED uses GPIO 21/22, the five LEDs use GPIO 25/26/27/32/33, and the demonstration servo signal uses GPIO 14. Figures 2 and 3 show the documented wiring. Photographs cannot establish concealed pin continuity, supply voltage, or resistor values.

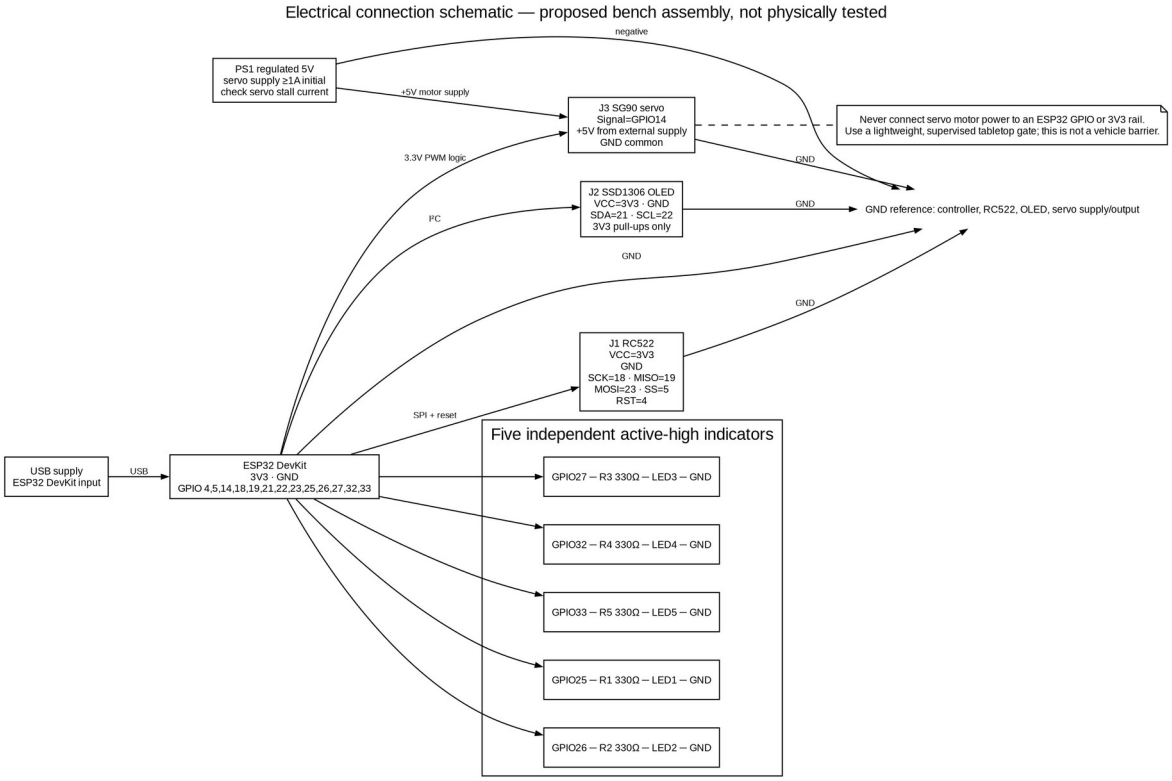


Figure 2. Proposed electrical wiring with separate servo power rail and a common ground reference.

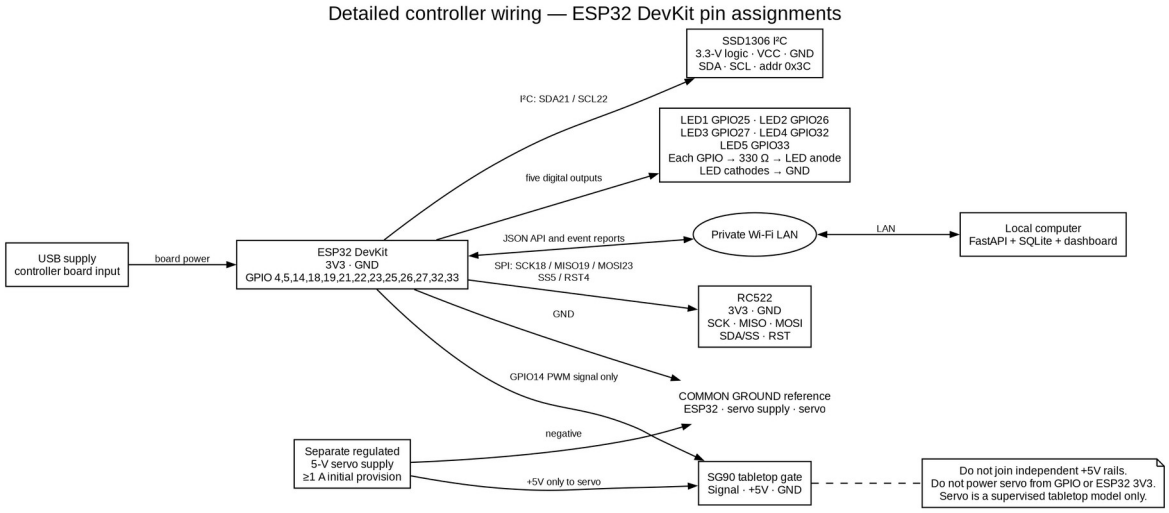


Figure 3. Proposed signal map for controller peripherals.

Table 3. Proposed controller pin assignments

Component	Signal	ESP32 pin	Voltage	Connection note
RC522	SCK / MISO / MOSI	18 / 19 / 23	3.3 V logic	VSPI; dedicated bus
RC522	SS / RST	5 / 4	3.3 V logic	Matches the current firmware pin definitions
OLED	SDA / SCL	21 / 22	3.3 V logic	Use compatible pull-ups; I²C address 0x3C
Slot LEDs 1–5	Anodes via 330 Ω	25, 26, 27, 32, 33	3.3 V outputs	Each LED has its own resistor; cathodes to GND
SG90	PWM signal	14	3.3 V signal	Servo +5 V from a separate regulated supply; common GND

The RC522 uses the firmware's SPI pins, and the OLED uses its I²C pins. Each of the five active-high LED outputs has its own series resistor. GPIO and boot-strapping constraints are documented in the ESP32 datasheet [18]. Table 3 follows the firmware pin definitions; the photographs do not independently verify each connection.

Power the controller through USB and supply 3.3 V logic to the RC522 and OLED. Use a separately regulated 5 V supply for the servo, with its ground connected to the controller ground; do not connect the servo's positive supply to the controller logic rail. Fit a 330 Ω resistor in series with each LED. Servo startup or stall current can exceed the capacity of a GPIO or development-board regulator, so size the external supply for the specific SG90 variant. Keep the lightweight model supervised. A hobby servo is not a safety-rated barrier actuator, and a power loss can leave its arm in place.

The bill of materials, per-signal pin map, wiring sequence, and troubleshooting guide are in docs/hardware/. The 26 October controller log records two RFID transactions, the change in S1 status, and LED and gate commands. LED physical output was not visually verified; servo movement was not independently observed, and supply current was not measured.

6. Methodology: Software and Machine-Learning Design

Table 4. Software components and responsibilities

Technology	Role in implementation
Python 3.14.8	Backend, data preparation, benchmarking, report tooling on the measured host.
FastAPI and SQLite	HTTP contracts, RFID lifecycle state, session-derived slot occupancy, prediction/optimizer metrics and dashboard data.

Technology	Role in implementation
NumPy and Pillow	Feature construction and server-side forecasting model training and inference.
TensorFlow/Keras	Server-side LSTM, GRU, and temporal Transformer regression models.
Arduino framework + PlatformIO	ESP32 controller build; device attachment is reported separately from compilation.
Graphviz and python-docx	Editable architecture diagrams and generated single-column DOCX/PDF submission.

6.1 Dataset and Preprocessing

The dataset is synthetic and deterministic. It contains 4,032 five-minute intervals over 14 days, 27 input features, and 6,274 privacy-safe source observations. The timeline covers five configured spaces from 2026-08-01T00:00:00Z through 2026-08-14T23:55:00Z and uses seed 20260908. Inputs include occupancy and its change, rolling averages, time and weekday cycles, simulated weather and event indicators, vehicle counts, active sessions, aggregated vehicle attributes, and allocation context. Each example predicts the occupancy ratio one interval ahead.

For each prediction, the preprocessor supplies the preceding 12 intervals (60 minutes) and forecasts the next one. It splits the timeline in chronological order into 70% training, 15% validation, and 15% test before building sequences; the resulting counts are 2,810, 592, and 594. Feature means and scales are fitted on training data only. All three neural models receive the same PreparedDataset, and no RFID UID enters the model. This setup keeps later observations out of earlier inputs and prevents test data from influencing scaling.

The feature builder forward-fills eligible short gaps. Longer gaps remain missing, and examples containing them are excluded from training. Bounded, cyclic, and one-hot features follow the existing input contract; all models share the same continuous-feature scaler. Because the final interval has no following target, it is not used as a labeled example. This dataset supports repeatable software comparisons, but it cannot measure real parking demand, weather effects, or performance at another site.

Dataset files and metadata are in review_outputs/2026-10-15/dataset. A public, version-pinned ZIP download is available from GitHub: download simulated dataset

6.2 Forecasting Architectures

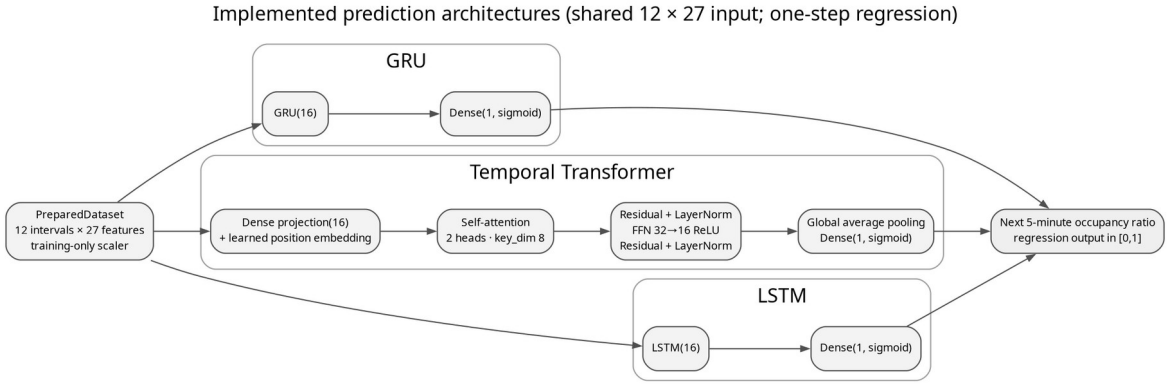


Figure 4. Implemented neural architectures; each receives the same scaled 12×27 sequence and predicts a bounded scalar.

The LSTM and GRU each use one 16-unit recurrent layer and a sigmoid output that predicts one occupancy value. For the Transformer, each 27-feature interval is projected to width 16 and combined with a learned positional embedding. Two-head self-attention (key dimension 8) feeds a residual connection and layer normalization, followed by 32- and 16-unit feed-forward layers, a second residual connection, global average pooling, and a sigmoid output. All models use mean-squared-error loss and Adam with a 0.001 learning rate. Training is capped at 12 epochs, uses batches of 32, and stops early after three validation epochs without improvement. TensorFlow/Keras deterministic operations are enabled where supported. Since occupancy is continuous, classification accuracy, F1, ROC curves, and confusion matrices do not apply.

Each Keras artifact is content-addressed and accompanied by a manifest recording feature order, split identity, scaler values, configuration, software versions, and SHA-256 hash. The server loads these artifacts for inference. TensorFlow Lite conversion, quantization, and pruning are outside the current design because the ESP32 does not run the neural models.

6.3 Allocation Objective

Each free candidate slot i receives the normalized cost $J(i) = 0.35\, d_i + 0.30\, c_i + 0.20\, w_i + 0.15\, f_i$. The terms represent distance, congestion, waiting, and fairness from the live allocation request. Occupied slots are removed before selection. For the benchmark, the application captures the request at each SUMO/TraCI parking admission and replays first-available, exhaustive search (the exact solution for this one-slot choice), GA, PSO, ACO, and NSGA-II against the same observed state and weights. For a heuristic choice x, its gap from the exact result is $J(x) - \min_i J(i)$, nonnegative apart from floating-point tolerance. The optimizer interface does not provide valid per-iteration histories, so convergence curves are not reported [14], [15].

The controller-side request sequence is:

```
read and normalize UID in RAM
if a saved operation exists: retry its same idempotency key
else: inspect current server status; choose entry or exit
persist operation + key in NVS before HTTP POST
validate response identity and gate_action; re-read status
if committed status matches: persist gate_done before one gate cycle
on timeout/conflict/malformed reply: keep gate closed
```

7. Implementation Details

The vehicle API accepts the JSON fields rfid_uid and idempotency_key. Before sending a request, the controller builds a key from its chip identifier and a sequence number stored in non-volatile storage, then saves the key with the pending operation. After a restart, it resends that same key and payload, allowing the server to return the result of an operation it has already committed. The firmware does not print tag identifiers to Serial. The API currently lacks device authentication and TLS, so the controller should only connect through a trusted private network.

The server and benchmark use the same weighted_cost function. If no space is free, the API returns HTTP 409 and the controller keeps the gate closed. Before opening it after entry, the firmware checks the returned tag, slot identifier and name, gate action, and current parking status. After exit, it confirms that the session is no longer active. The five LEDs show the accepted server state: on for occupied, off for free, and blinking together when status is unknown. The OLED reports startup, network state, scans, assignments, full-lot responses, completion, and errors. A timed state machine closes the demonstration gate after its configured interval.

A slot changes to occupied only when the server commits the RFID session and allocation. On exit, the same session is closed and its assigned slot becomes free. Both the LEDs and dashboard use the resulting server status, keeping their view of occupancy aligned.

8. Experimental Setup

The forecast comparison uses the generated synthetic timeline and its chronological split. Each neural model shares the same preprocessing and test partition; the persistence baseline carries the latest observed occupancy forward by one interval. Server inference latency is the median warm-run prediction time divided by the number of examples. The run used TensorFlow 2.22.0-rc0, Keras 3.16.0.dev2026101404, and Python 3.14.8 on Linux x86_64 with an Intel Core i7-14650HX. No accelerator or ESP32 inference time is included.

The allocation benchmark covers 120 SUMO-observed parking admissions, 3 optimizer seeds per observed state, and 30 iterations for each heuristic. It uses the configured weights {"congestion": 0.3, "distance": 0.35, "fairness": 0.15, "waiting": 0.2}. SUMO Eclipse SUMO sumo 1.28.0 ran with seed 20261031 and 0.25-second steps; configuration, network, and route SHA-256 hashes are included in the evidence. Candidate inputs are preserved in `evidence/sumo_observed_scenarios.jsonl`; results are replayed on each captured live-allocation request. Runtime is measured with a monotonic clock. Rejections at full capacity are counted separately. These results measure this modeled SUMO traffic run, not a real facility or measured queue delay.

The host has an Intel Core i7-14650HX, 24 CPUs online, and 15 GiB RAM. Model and allocation runtimes were measured in software on this machine and may vary with system load. For the hardware trial, an ESP32 DevKit connected to the local API over Wi-Fi. The results below come from the controller's timestamped serial capture and the API access log dated 26 October 2026.

9. Results and Discussion

9.1 Prediction Results

Table 5 reports the persistence baseline and three neural regressors on the same chronological test windows. GRU has the lowest MAE and RMSE in this run. The ranking applies to one synthetic dataset and seed; it does not establish that GRU will perform best on other data or at a real parking site. Inference times are from the server CPU, and model sizes refer to the serialized Keras files.

Table 5. Test-set prediction performance by model

Model	MAE	MSE	RMSE	Latency ms/example	Size bytes
persistence_baseline	0.120875	0.028620	0.169173	0.000015	N/A
lstm	0.104062	0.016818	0.129685	0.245910	60481
gru	0.098055	0.013891	0.117862	0.223783	52615
transformer	0.109173	0.018098	0.134531	0.232451	110684

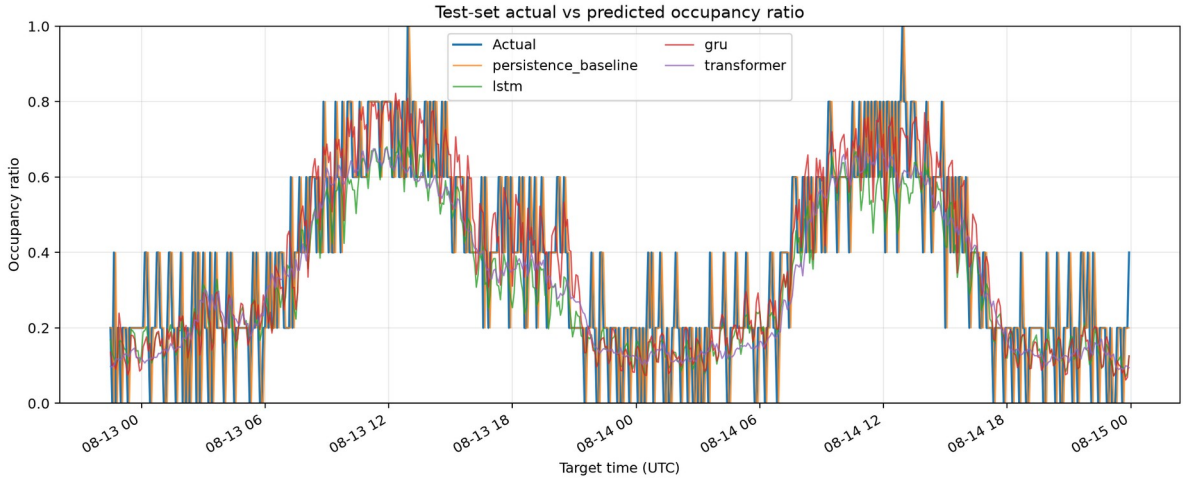


Figure 5. Test-window actual versus predicted occupancy ratio from the executed model comparison.

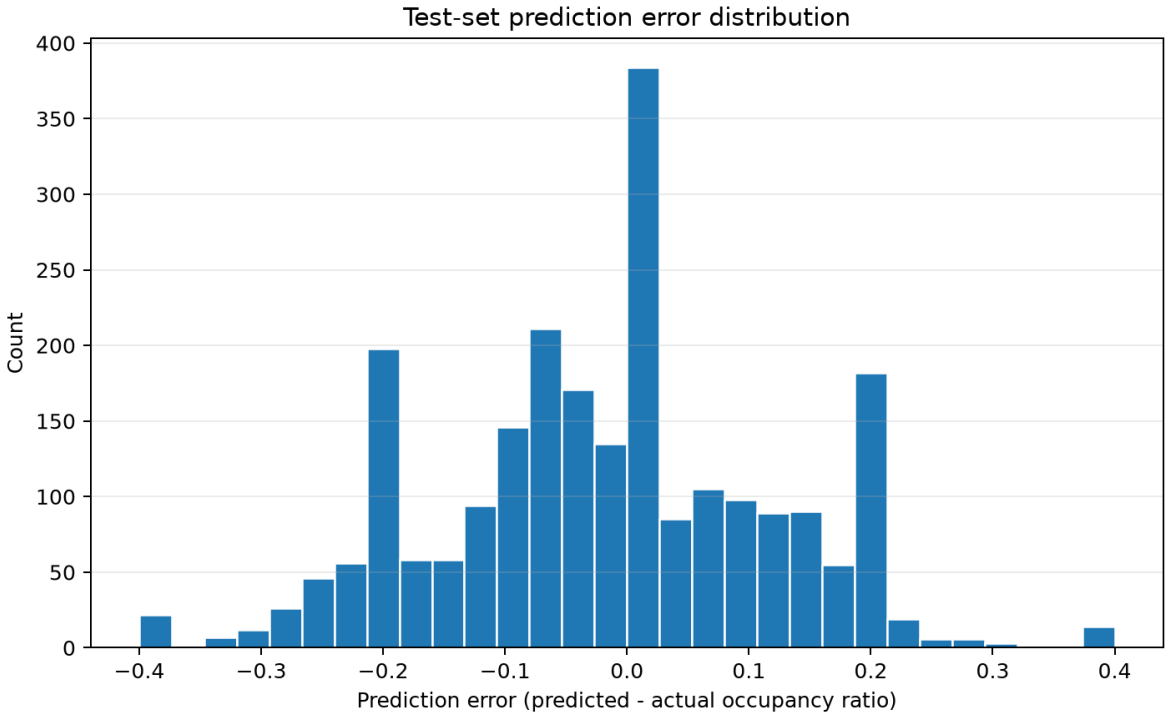


Figure 6. Prediction error distribution; errors are regression residuals, not class labels.

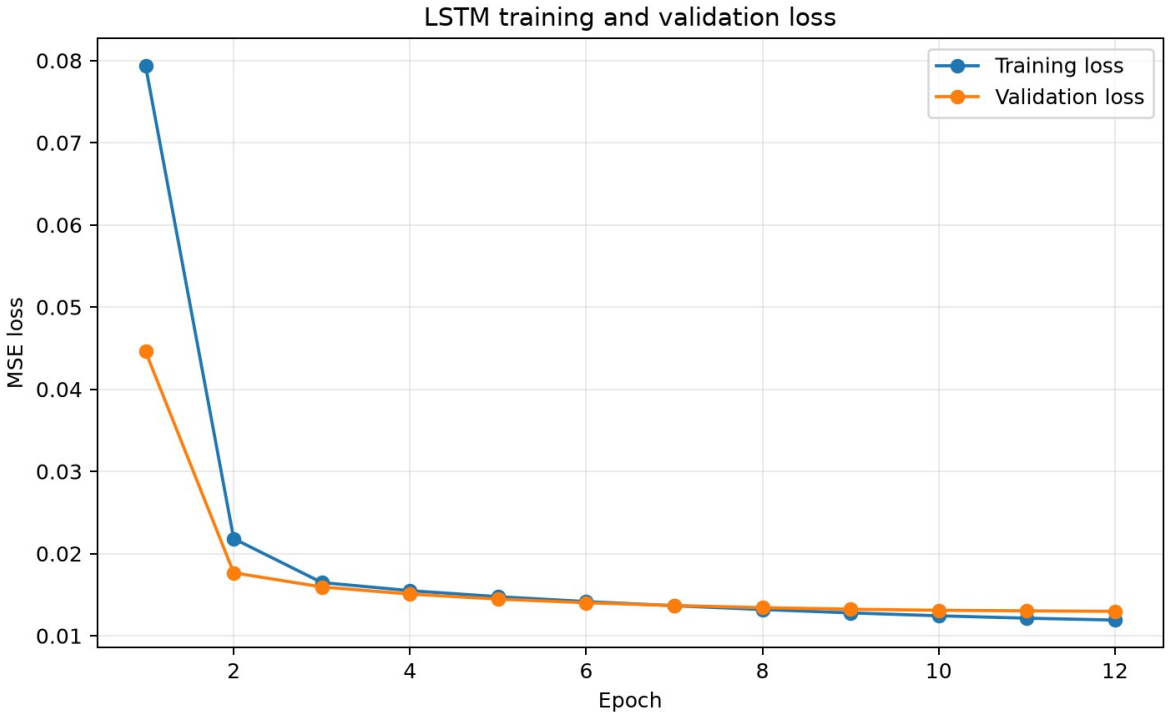


Figure 7. LSTM training and validation loss for the fitted run.



Figure 8. GRU training and validation loss for the fitted run.

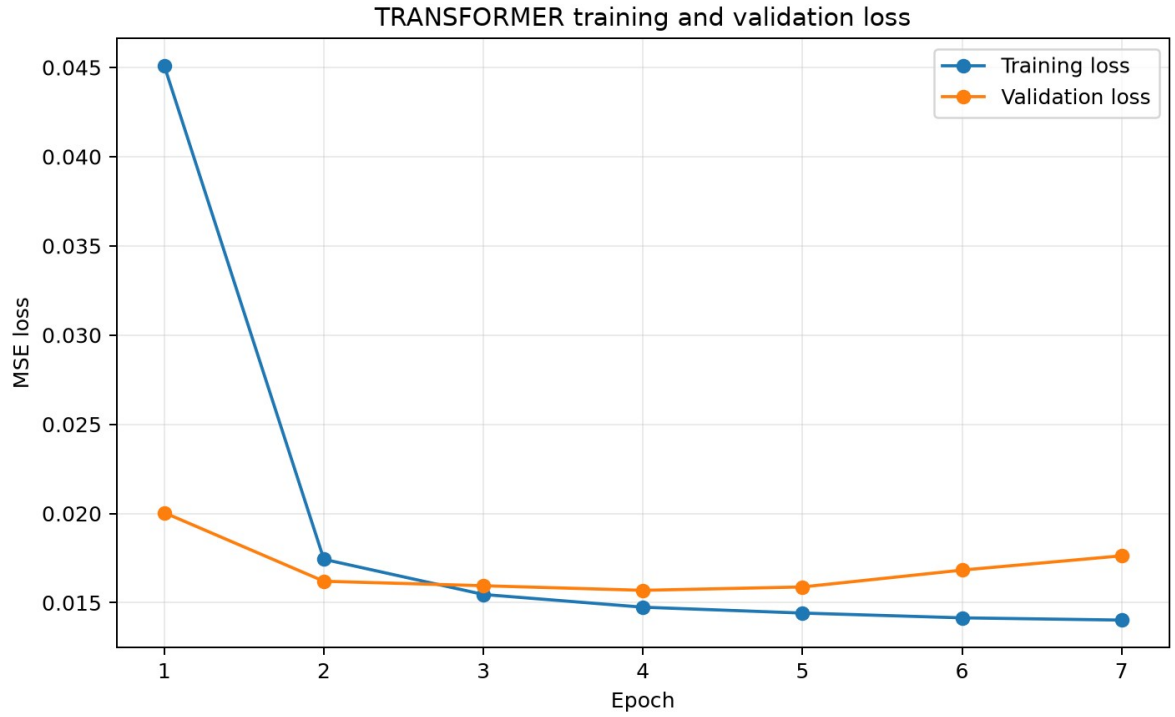


Figure 9. Transformer training and validation loss for the fitted run.

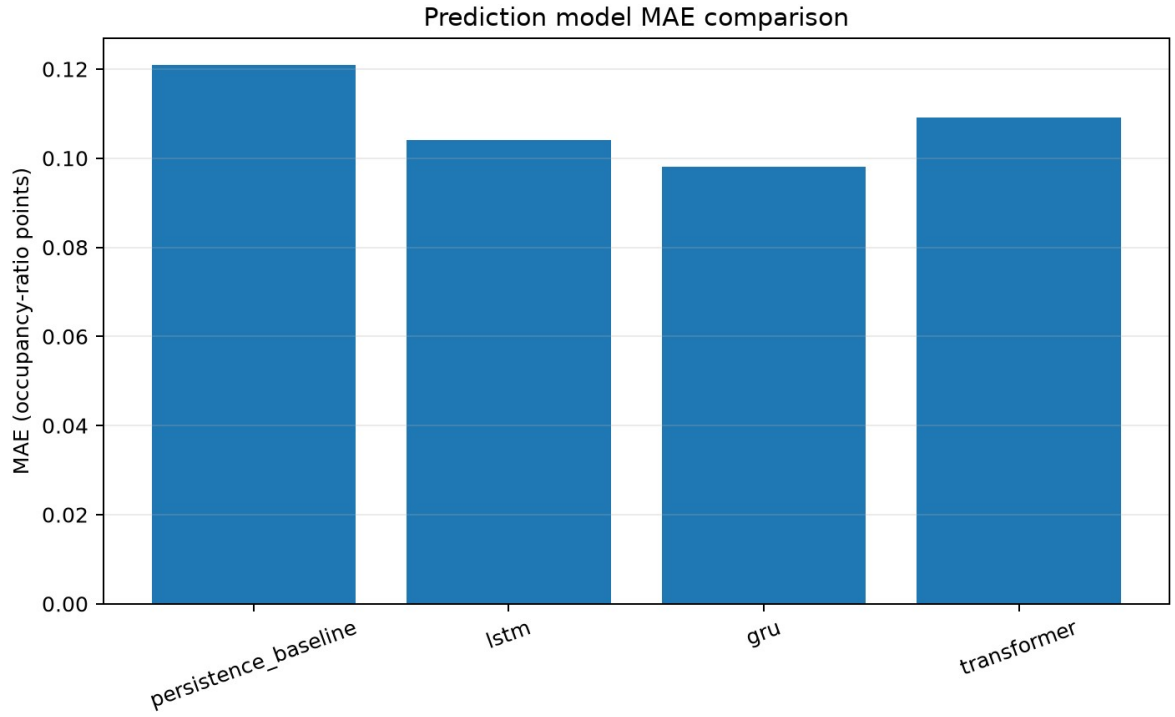


Figure 10. Measured model MAE comparison on the shared chronological test partition.

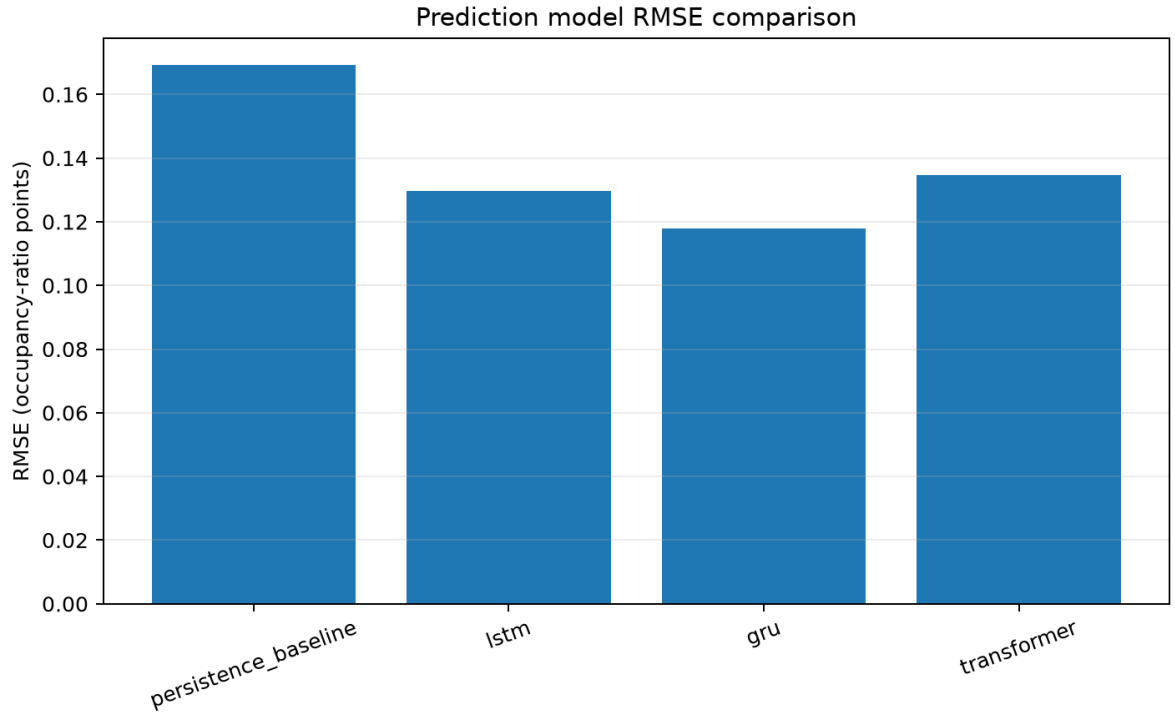


Figure 11. Measured model RMSE comparison on the shared chronological test partition.

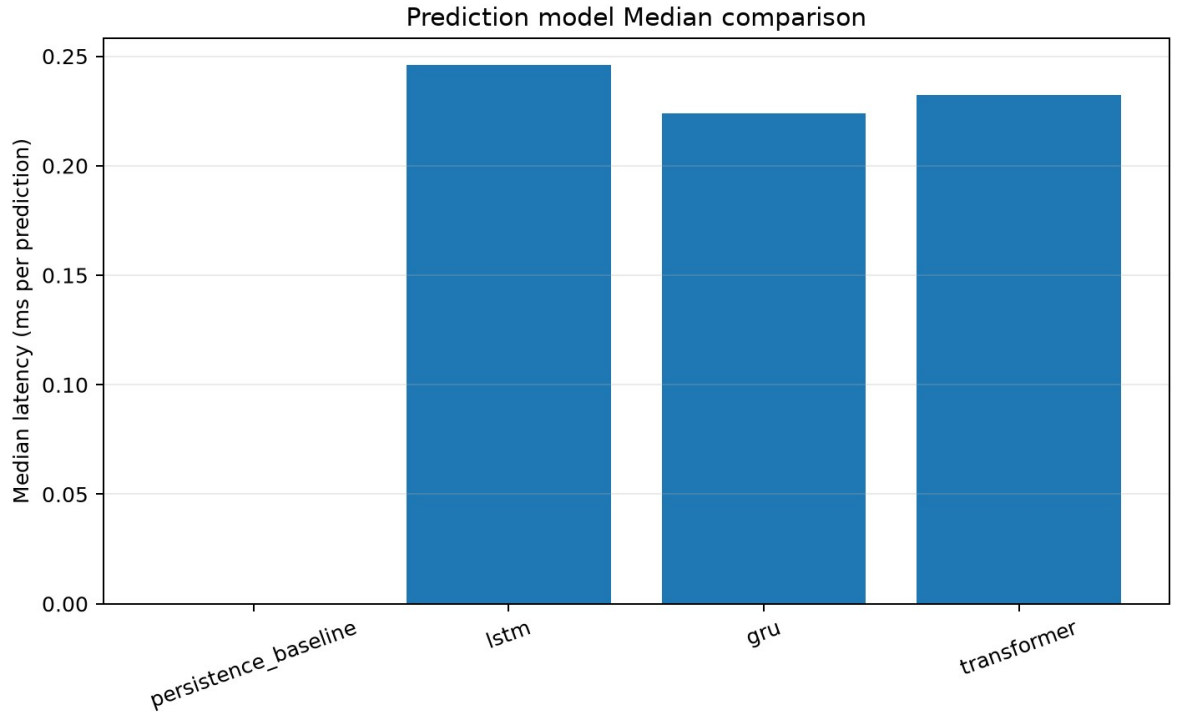


Figure 12. Measured per-example inference latency on the server host.

On the shared synthetic test set, GRU lowers MAE by 18.9% and RMSE by 30.3% relative to persistence. LSTM is second on both measures. The Transformer also improves on persistence, but not as much as the recurrent models in this run. Some of the 27 inputs, including simulated calendar, weather, and event context, may reflect patterns specific to the generator. Measured occupancy data, evaluation at another site, and independently seeded datasets are needed before making deployment claims.

9.2 Allocation Results

Table 6 aggregates 120 parking allocation states observed during the executed SUMO run and 3 optimizer-seed repetitions per state across 2,376 candidate bays. Each policy rejected 0 full-capacity attempts. Exhaustive search gives the exact minimum for this one-slot decision, with mean cost 0.174159. GA mean cost was 0.177172; NSGA-II mean cost was 0.176095, with a mean optimality gap of 0.001936 and median runtime 783.795 ms. Cost and runtime are measured from these captured modeled states and Python optimizer executions; they do not establish real-facility throughput or field performance.

Table 6. Allocation performance across all repeated runs

Policy	Success / attempts	Mean cost	Median runtime ms	Mean gap	Invalid
first_available	360 / 360	0.184562	0.000772	0.010403	0
exhaustive_optimum	360 / 360	0.174159	5.346467	0.000000	0
GA	360 / 360	0.177172	0.584910	0.003012	0
PSO	360 / 360	0.177453	0.885997	0.003294	0
ACO	360 / 360	0.176658	159.144138	0.002498	0
NSGA-II	360 / 360	0.176095	783.795228	0.001936	0

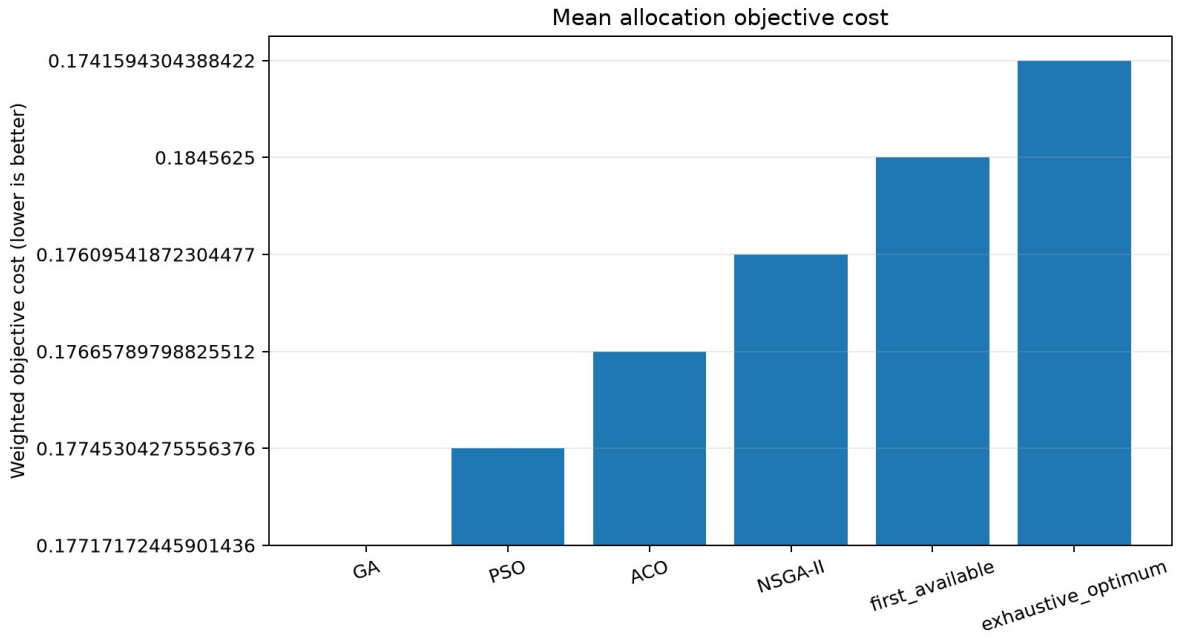


Figure 13. Mean allocation objective cost across all six policies, including NSGA-II, at 2,376 modeled bays.

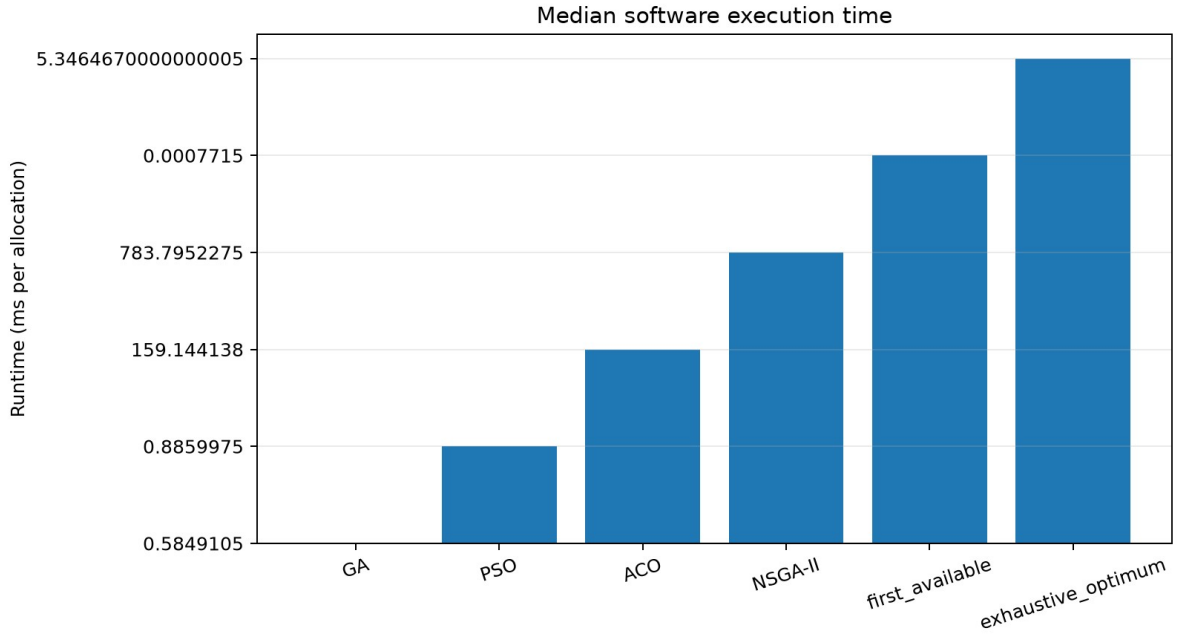


Figure 14. Software execution-time comparison; logarithmic scaling is labeled in the figure.

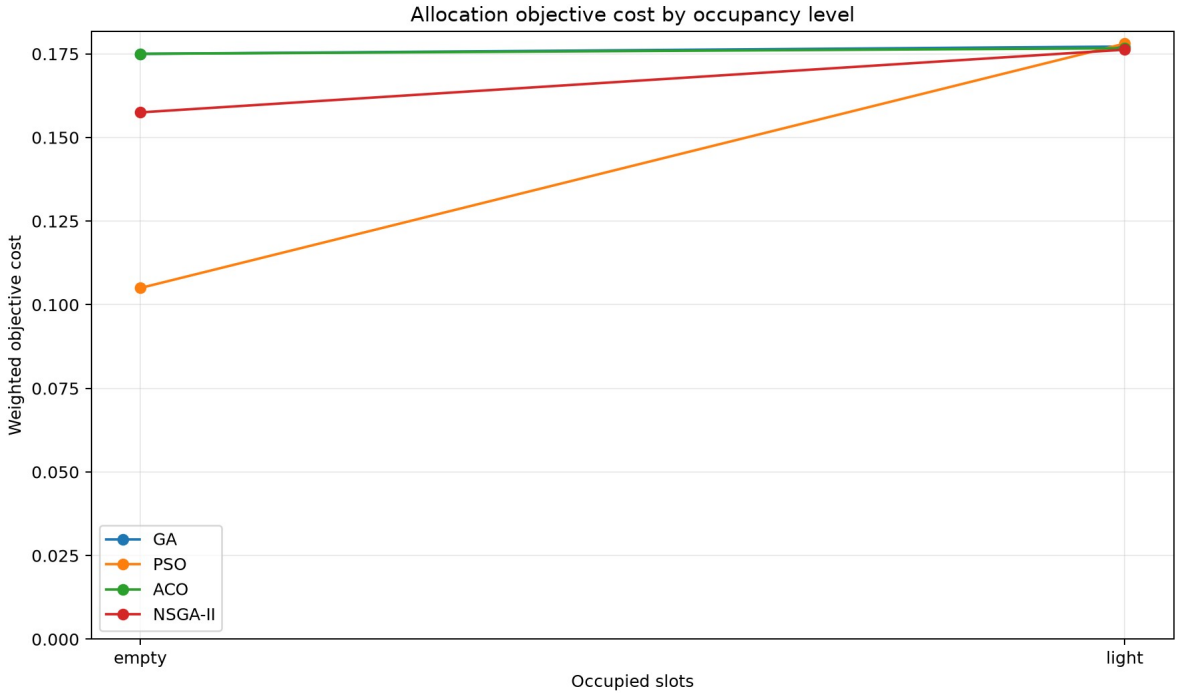


Figure 15. Objective cost by simulated occupancy category.

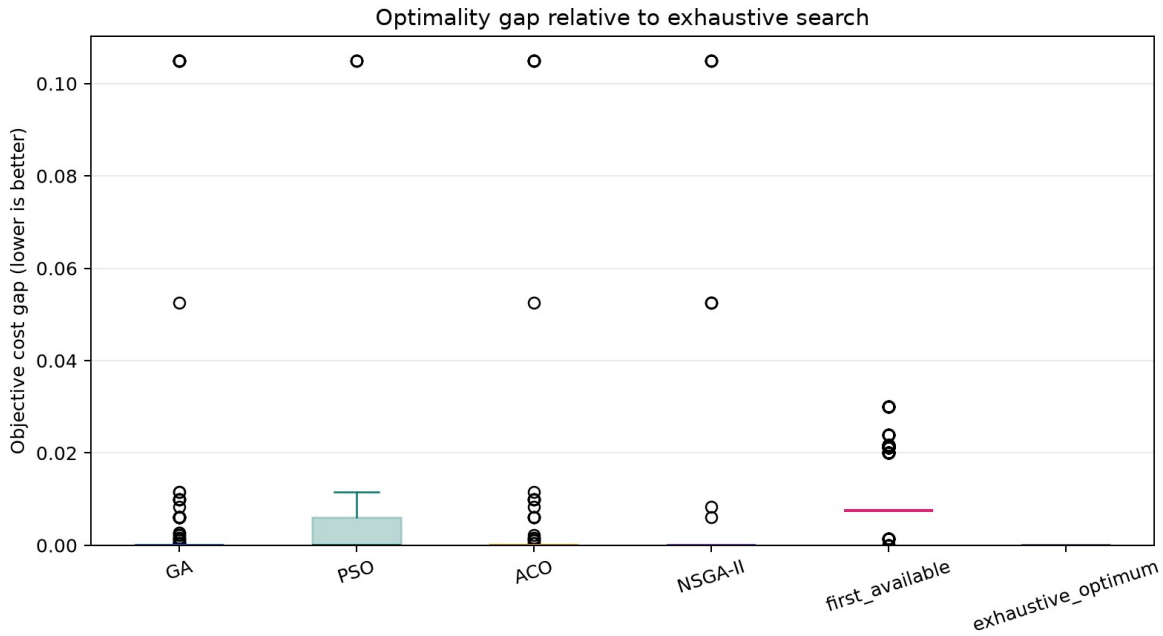


Figure 16. Per-run nonnegative gap from exact enumeration.

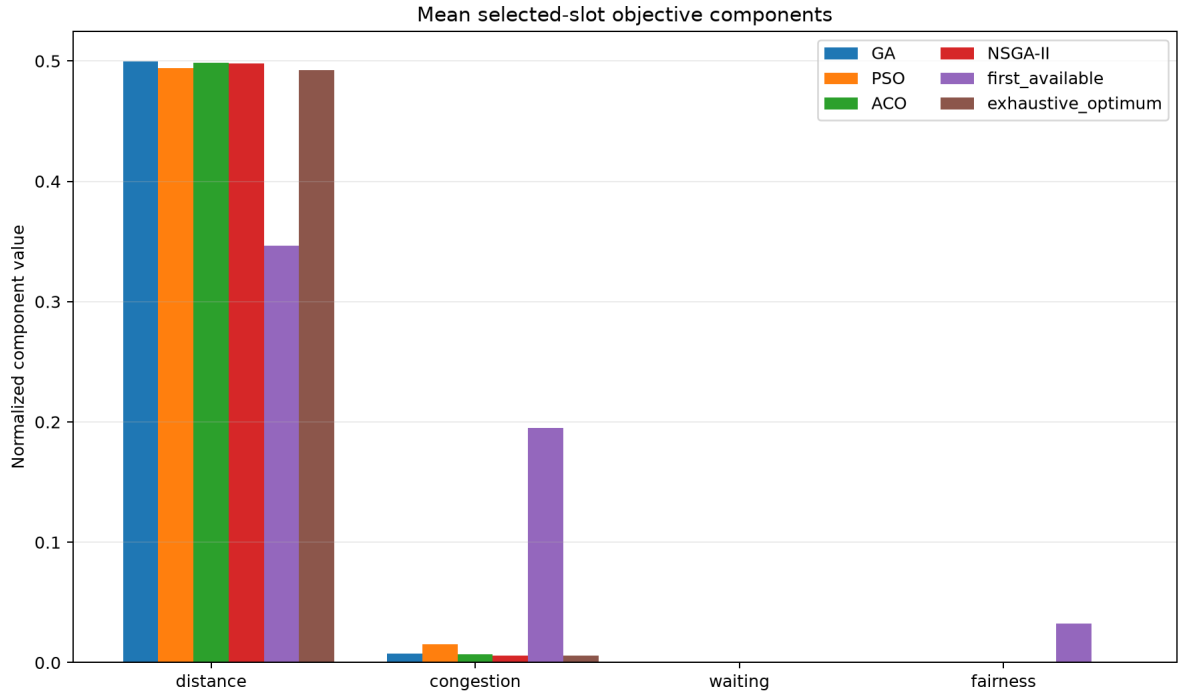


Figure 17. Mean weighted objective components among successful allocations.

Table 7. Mean objective cost by SUMO-observed occupancy category

Occupancy category	First available	Exact	GA	PSO	ACO	NSGA-II
empty	0.070000	0.070000	0.175000	0.105000	0.175000	0.157500
light	0.185525	0.175035	0.177190	0.178062	0.176672	0.176252

Table 7 includes only occupancy categories observed during the SUMO run (empty: 3 runs per policy; light: 357 runs per policy). No results are generated for unobserved occupancy levels. Full-capacity rejections (0) are reported separately. Runtime measures the Python algorithms on this host; it does not represent measured facility throughput or real queue waiting time.

9.3 Hardware and integration test status

PlatformIO compiled both firmware targets. The controller was identified as ESP32-D0WD-V3 revision 3.1; esptool verified the hash of the flashed firmware binary, and an earlier full flash backup is retained. During the logged run, serial telemetry recorded 3 boots and 3 Wi-Fi connections. The API log showed 5 successful health checks and 5 authenticated parking-status responses after the latest boot. The post-rewire trial detected 2/2 test scans; both entry and exit transactions returned HTTP 200. Entry transaction latency: 1431 ms. Exit transaction latency: 1220 ms. Server status showed S1 changed to occupied at entry and free at exit. Across 8 serial events, free heap averaged 237202 bytes (median 237304, standard deviation 1027, p95 238352). Four gate-command timestamps averaged 4611 ms (median 4550, p95 6144); these are command timestamps, not measured servo travel. The LEDs were not visually checked. Current draw, Wi-Fi recovery time, and physical servo movement remain NOT TESTED. Historical isolated RC522 VersionReg probes returned 0xFF and 0x00; these were not repeatability trials and are reported separately from the later successful RFID transaction observations.

Table 8. Verification evidence and physical test status

Check	Observed result
Controller firmware build	PASS: latest controller firmware was flashed to ESP32-D0WD-V3 revision 3.1; esptool binary hash verification passed. An earlier full flash readback is retained.
USB serial/network reachability	PASS: 2/2 post-rewire vehicle transactions returned HTTP 200 after slot-status synchronization.

Check	Observed result
RC522 reader and card scan	PASS: 2/2 post-rewire RFID test scans succeeded; entry and exit both completed with HTTP 200. This two-trial result is not a reliability estimate.
OLED, LEDs, servo	The firmware logged five LED states and commanded S1 on at entry and off at exit. LED physical output was not visually verified. Servo values are command timing; physical movement was not independently measured.
ESP32 inference latency / memory	Not applicable to the current server-side TensorFlow design; not measured on device.

Table 9. Recorded physical measurements and untested quantities

Metric	Observed result	Interpretation
RFID reads	2/2	Two operator-confirmed scans; descriptive only, not a reliability estimate.
Entry / exit API transactions	2/2	Both returned HTTP 200 after controller status synchronization.
Entry response latency	1431.0 ms (n=1)	One measured transaction; no population latency claim.
Exit response latency	1220.0 ms (n=1)	One measured transaction; no population latency claim.
Free heap telemetry	mean 237202; median 237304; p95 238352 bytes (n=8)	ESP32 serial telemetry during the logged run.
Gate command delay	mean 4611; median 4550; p95 6144 ms (n=4)	Firmware command timestamps; servo arm travel was not measured.
Wi-Fi recovery / current / visual LED correctness	NOT TESTED	No controlled outage or electrical measurement was captured.

Both recorded RFID scans were accepted, and both API transactions succeeded. Entry response time was 1,431 ms; exit response time was 1,220 ms, with one observation in each direction. The 211 ms difference is descriptive, not a comparison of typical performance. Table 9 also reports gate-command timestamps and free-heap samples. A command timestamp does not measure how quickly the servo arm moves. Current draw, visible LED operation, and Wi-Fi recovery were not measured.

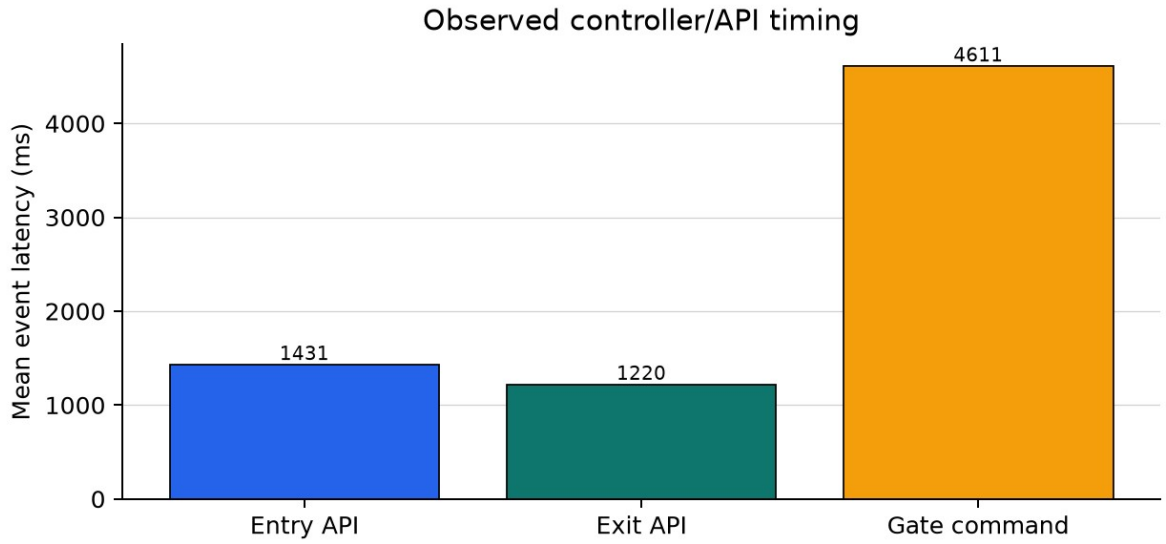


Figure 18. Observed entry, exit and gate-command event timing; command latency is not servo travel time.



Figure 19. 28 October scene capture of the fictional Parkflow Central district and surrounding city traffic; a modeled setting, not a current Phoenix Marketcity map.

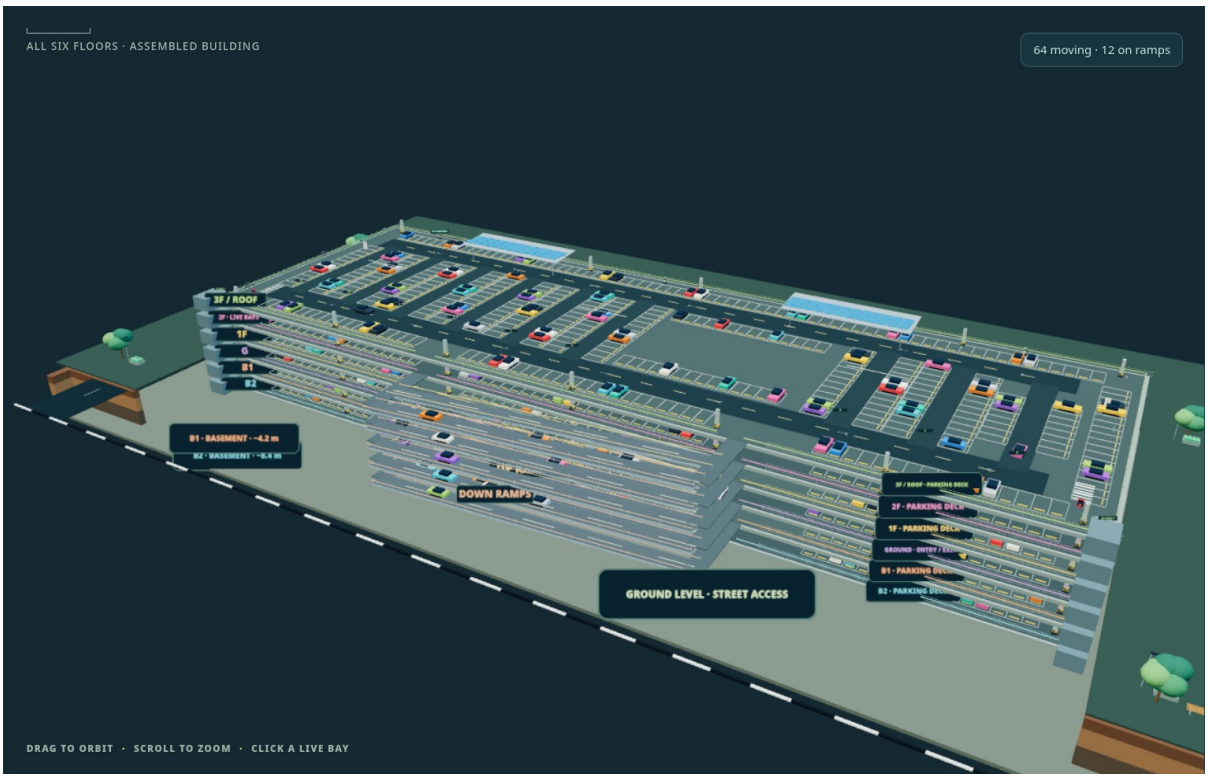


Figure 20. 28 October scene capture of the assembled six-deck garage, including two below-ground levels, four above-ground levels, connecting ramps, and animated traffic.

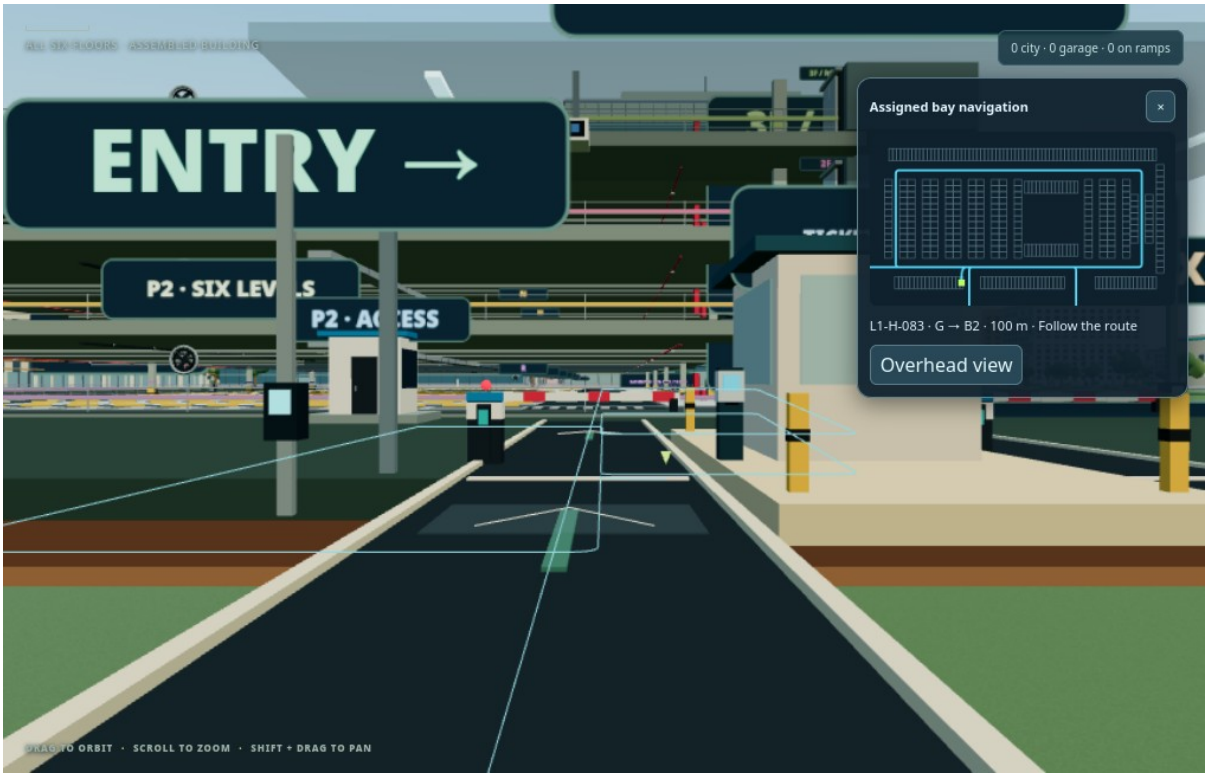


Figure 21. 28 October first-person entrance capture with the assigned bay, floor transition, route distance, and animated route guidance.

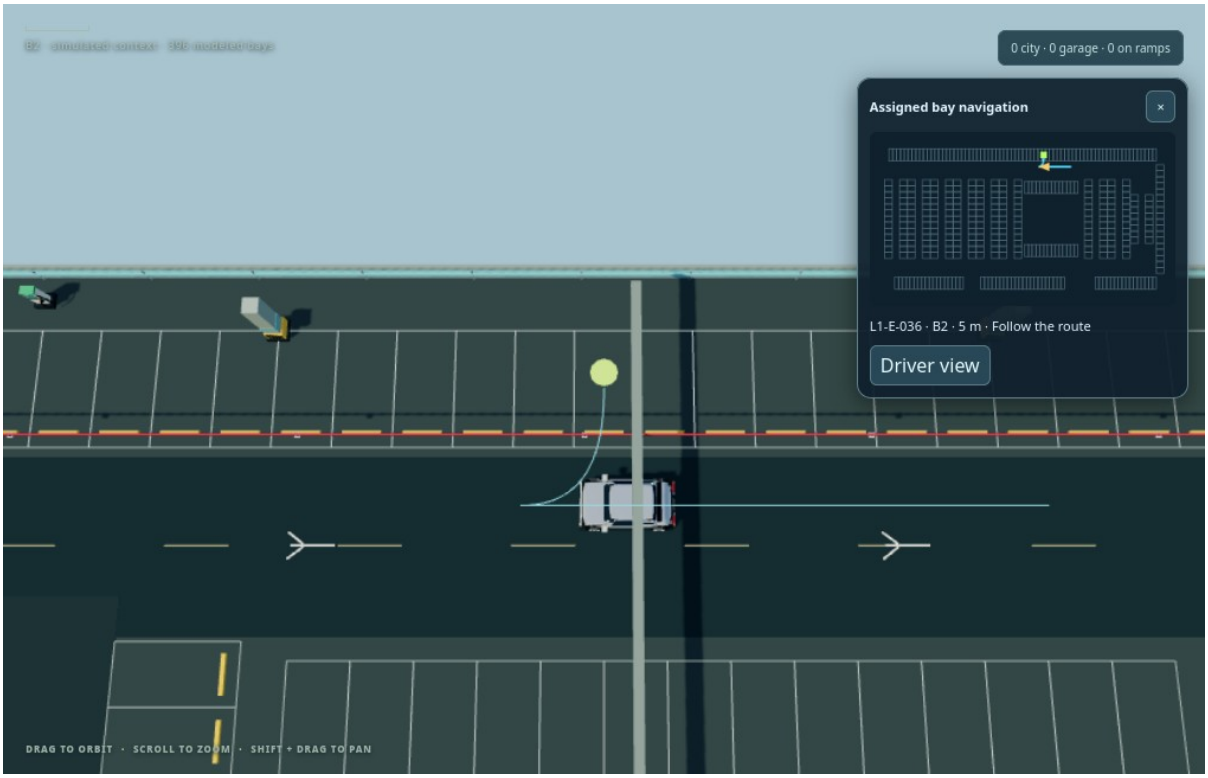


Figure 22. 28 October overhead parking-map capture showing the route, current vehicle position, and destination bay on its floor.

Figures 19–22 are scene captures from 28 October 2026. They document the city, multi-floor garage, first-person driver view, and bay navigation overlay. The screenshots show modeled traffic and geometry; they are interface evidence, not measured traffic outcomes. Tables 8 and 9 report the separate five-space physical-controller evidence. Hardware photographs are omitted.

9.4 Source- and specification-based engineering estimates

Table 10 gives planning estimates for quantities that were not measured. They are engineering assumptions, not hardware measurements or benchmark results. The read-rate and latency ranges assume a close-range tabletop setup on stable local Wi-Fi; component timing and current ranges draw on firmware settings and manufacturer specifications where available. The two recorded scans are functional observations only and do not establish a success rate.

Table 10. Rough engineering estimates (not measured)

Quantity	Estimate	Basis and limitations
RFID successful reads per close, aligned tap	Measured 2/2 scans; not a reliability estimate	Two operator-confirmed close-range test taps were detected. The sample is too small to estimate reliability; MFRC522 specifies up to 50 mm depending on antenna tuning, not a read-success guarantee [17].
API entry/exit transaction success on stable local LAN	Measured 2/2 transactions; not a reliability estimate	Two post-rewire entry/exit trials returned HTTP 200. This small sample does not estimate long-run availability.
LED correctness with known slot state	NOT MEASURED; firmware output commands logged	Serial telemetry recorded the five free-slot commands, S1 on at entry, and S1 off at exit. No visual or electrical output sensor was used.
Entry or exit latency, tag detection to accepted API response	Measured entry 1431 ms; exit 1220 ms (n=1 each)	Host-timestamped ESP32 serial events; one trial per direction is descriptive only. The 3.5 s HTTP timeout is a failure limit, not expected latency.
Wi-Fi recovery after AP becomes available	5–15 s (low confidence)	Firmware retries every 5 s; range allows up to about 10 s for association/DHCP after a retry. Poor AP conditions may take longer or fail.
Servo command delay, RFID detection to authorized GPIO command	Measured command events: 3300 ms, 5800 ms, 3200 ms, 6144 ms	Firmware command timestamps include gate-open and gate-close commands; they do not measure servo shaft travel or barrier motion.
SG90 travel for configured 10° → 85° sweep	0.13–0.50 s (very low confidence)	Ideal nominal is 75° × 0.1 s/60° ≈125 ms at 4.8 V; upper range is a load/settling allowance. Exact servo variant is unverified [18].
Controller free heap	Measured mean 237,202 bytes (n=8)	Current serial capture: median 237,304 bytes, SD 1,027, p95 238,352. Measured telemetry, not a future guarantee.
Controller + reader + OLED + LEDs input current, servo excluded	0.15–0.40 A at 5 V (very low confidence)	Budget estimate from ESP32 active Wi-Fi radio draw plus peripherals; regulators, OLED brightness, LED current, and network duty cycle vary [18].
SG90 servo supply current	0.5–2.0 A (very low confidence)	Manufacturer product page notes this operating-current range; exact unit/load is unknown. Use a separate regulated servo supply, never ESP32 GPIO or 3.3 V [18].

9.5 Current full-scale software implementation

This update records the application state on 31 October 2026 and supplements the earlier experimental results in this report. The dashboard presents a six-level parking-garage emulation connected to a surrounding fictional city district and a native SUMO/TraCI road simulation. Its 2,376 bays are all represented in the garage map and occupancy records: 396 per deck, with two below-ground decks and four above-ground decks connected by ramps. The layout uses published Haneda P2 floor guides as a geometric reference; the basement placement, building, and city are emulations, not an exact Phoenix Marketcity or Haneda map. The rendered city adds roads, buildings, planted areas, trees, pedestrians, street lighting, and traffic signals, with animated vehicles, barriers, and ramps. The browser refreshes SUMO vehicle snapshots and the parking event stream, keeping movement and bay occupancy tied to current server state. The five-space RFID rig remains the physical prototype; the larger garage is software-modelled.

The dashboard provides SUMO start, stop, pause, resume, and reset controls and reports the current run state, simulated time, vehicle count, and mean speed. Traffic vehicles that reach the garage approach can enter the parking workflow: each SUMO vehicle ID maps to a stable pseudonymous parking identity across simulation runs, while entry, exit, and rollback operations retain run-specific idempotency keys. The system records a parking session, removes the vehicle from road traffic while parked, then closes the session and rejoins it to traffic on departure. The model caps admission at two arrivals per simulation step and uses a deterministic emulated dwell interval based on the assigned bay and floor; this is a controlled traffic/parking emulation, not a measured arrival rate or throughput result. Separate entry and exit approaches, gate arms, routes, and garage barriers are visible in the model.

Drivers can use the manual vehicle-ID simulator or an RFID/keyboard reader; the connected ESP32 scan and simulated ID feed the same entry/exit lifecycle. Scanning a known active identity selects exit, while an identity with no active session selects entry. An RFID UID is normalized to uppercase and stored as a unique vehicle identity. Each identity may have only one active session; repeat visits create separate records with their own entry and exit times. On entry, the assigned vehicle is animated from the entrance toward its selected bay, with first-person view, floor/route status, distance, and a map waypoint. The same bay appears in live occupancy and can be followed again from the receipt. Exit closes the session and animates departure. Every one of the 2,376 modeled bays has an occupancy record, although only the five RFID spaces are wired to the physical prototype.

The emulated car tariff is ₹50 for the first started hour and ₹40 for each additional started hour, with a ten-minute grace period and a ₹400 cap per 24 hours. Charges are computed per session from its entry and exit timestamps and stored in paise. The tariff is illustrative; it is not presented as the mall's verified official rate. Entry and exit return a session receipt with vehicle identity, bay, timestamps, duration, tariff, and live estimate or final amount. The dashboard displays a QR code that opens the full receipt on a phone. This is a receipt link, not QR-based payment; no payment gateway or settlement is implemented. The same tariff rule applies to every identity, while session duration makes each charge specific to that vehicle's visit.

NSGA-II is integrated into live slot allocation and the optimizer comparison API, configured with population 48, 60 generations, crossover 0.90, and mutation 0.12. It selects a weighted compromise from a non-dominated candidate set. The benchmark compares it with GA, PSO, ACO, first-available, and exhaustive search by replaying requests captured at parking admissions during an executed SUMO/TraCI run across all 2,376 modeled candidates. It records the SUMO version, seed, network and route hashes, simulation times, raw candidate states, and Python optimizer runtimes. The allocation outcomes are actual measurements from this fictional-garage simulator run, not random objective scenarios or physical mall results. Live allocation computes personalized fairness from each normalized vehicle identity's completed sessions: for each candidate bay, it compares that driver's projected mean walking distance after assignment with the equal-weighted mean of each other vehicle's historical mean walking distance across its completed sessions. The absolute difference is the fairness cost, capped to the normalized 0–1 range; lower cost favors assignments that reduce disparity. With no completed peer history, fairness is neutral at zero. Stable SUMO identities let repeat simulated vehicles retain this history across runs. Waiting remains zero because the model has no measured queue-wait estimate. This is a software allocation proxy based on configured walking distances, not a guarantee of equitable outcomes. The tariff remains identity-neutral. The full software test suite passed 249 tests. No new physical RFID, QR-reading, payment-settlement, gate-actuation, or SUMO-throughput trial is claimed.

10. Testing and Demonstration

Automated software checks cover lifecycle retries and conflicts, full-lot handling, state persistence, prediction fallback and artifact compatibility, and simulated API ordering. The build checklist reports these results separately from firmware compilation and hardware observations. The post-rewire capture records two RFID reads, two successful HTTP 200 transactions, S1 occupied at entry and free at exit, and four gate-command timestamps. These two trials demonstrate the transaction flow; they do not establish reliability. LED output, servo movement, and current draw were not independently measured. Slot occupancy is derived from active RFID sessions.

A host-runtime check copied the review database, selected the saved GRU artifact in the predictor setting, and called GET /api/prediction. The API returned model=gru, fallback_reason=null, and occupancy_ratio=0.136493 for the synthetic timeline. This confirms that the server can load the artifact and return a prediction; it is not a controller measurement.

In a separate software-only run, /health reported five configured spaces, a simulated entry was assigned S1, and the API returned GA, PSO, ACO, and NSGA-II benchmark results for the same scenario. Because the isolated database contained too little history, prediction used the expected deterministic fallback, insufficient_history. These results check software behavior; the simulated tag does not count as a physical RFID read.

To repeat the software demonstration, activate the project environment, run `make test` and `make lint`, and start the API with `make run`. Then run `python -m scripts.run_simulated_demo --base-url http://127.0.0.1:8000 --rfid-uid 04A1B2C3 --idempotency-key final-review-entry-001` and open http://127.0.0.1:8000/dashboard. The experimental report and figures are in `review_outputs/2026-10-15/EXPERIMENTAL_REPORT.md` and `review_outputs/2026-10-15/final_submission/figures/`. Label this as a simulated RFID workflow. Avoid `make review` during the presentation because it retrains the models and regenerates result files.

For the controller demonstration, first confirm GET /health. Scan a dedicated test tag for entry, check the assigned space and controller status, then scan again and confirm that the space is free. Keep fingers clear of the supervised tabletop servo. DEMO_INSTRUCTIONS.md contains the startup, cleanup, and failure-recovery steps.

11. Limitations and Future Work

The main limitations are the synthetic forecasting dataset, optimizer results from one SUMO-generated traffic run, configured rather than site-measured bay distances, zero queue-wait values, a five-space physical RFID rig alongside a modeled 2,376-bay garage, a one-step forecast horizon, patterns specific to the forecast data generator, and two RFID/API trials. The allocation benchmark is actual output from the executed simulator, but it is not a physical mall trial. Firmware logs show LED commands, but the LEDs were not checked visually. Servo travel and power draw were also not measured. Synthetic forecast accuracy cannot establish how the models will perform at an operating site.

The next evaluation should use measured occupancy timelines, test separate days and sites, and repeat the model runs with independent seeds. Hardware work should include more RFID/API trials, direct checks of all five LEDs and servo movement, and measured current and Wi-Fi recovery time. Device requests also need transport authentication before the system is exposed beyond a trusted local network. A vehicle barrier requires a safety-rated actuator, controller, and independent interlocks; the SG90 model is only a tabletop demonstration.

12. Conclusion

This project brings together RFID entry and exit, session-based occupancy, space allocation, server-side forecasting, a browser dashboard, and ESP32 controller firmware. In the synthetic forecast test, GRU had the lowest MAE of the three neural models; for optimizer benchmarking, exhaustive enumeration provided an exact reference for each SUMO-observed allocation state. The tabletop trial completed two entry/exit transactions and recorded the change in S1 status along with API and gate-command timings. The small sample demonstrates that the components can work together, but it does not establish deployment reliability. Physical LED output, servo travel, current draw, and Wi-Fi recovery time still need direct measurement.

References

[1] S. S. Channamallu, S. Kermanshachi, J. M. Rosenberger, and A. Pamidimukkala, “Parking occupancy prediction and analysis – a comprehensive study,” Transportation Research Procedia, vol. 73, pp. 297–304, 2023. doi: 10.1016/j.trpro.2023.11.921.

[2] S. Bilotta, L. A. Ipsaro Palesi, and P. Nesi, “Predicting Free Parking Slots via Deep Learning in Short-Mid Terms Explaining Temporal Impact of Features,” IEEE Access, vol. 11, pp. 101678–101693, 2023. doi: 10.1109/ACCESS.2023.3314660.

[3] F. Zhang, K. Shang, L. Yan, H. Nan, and Z. Miao, “Prediction of Parking Space Availability Using Improved MAT-LSTM Network,” ISPRS International Journal of Geo-Information, vol. 13, no. 5, article 151, 2024. doi: 10.3390/ijgi13050151.

[4] F. J. Enríquez, J.-M. Mejía-Muñoz, G. Bravo, and O. Cruz-Mejía, “Smart Parking: Enhancing Urban Mobility with Fog Computing and Machine Learning-Based Parking Occupancy Prediction,” Applied System Innovation, vol. 7, no. 3, article 52, 2024. doi: 10.3390/asi7030052.

[5] A. Dahiya, P. Mittal, Y. K. Sharma, U. K. Lilhore, S. Simaiya, E. Ghith, and M. Tlija, “Machine Learning-Based Prediction of Parking Space Availability in IoT-Enabled Smart Parking Management Systems,” Journal of Advanced Transportation, 2024, article 8474973. doi: 10.1155/2024/8474973.

[6] H. Kuang, K. Deng, Q. Wang, W. Ye, H. Qu, and J. Li, “Deep meta-learning approach for regional parking occupancy prediction considering heterogeneous and real-time information,” Advanced Engineering Informatics, vol. 64, article 102969, 2025. doi: 10.1016/j.aei.2024.102969.

[7] A. Kaltenbrunner, J. Ferrer, D. Moreno, et al., “A simple and explainable model for park-and-ride car park occupancy prediction,” EPJ Data Science, vol. 14, article 74, 2025. doi: 10.1140/epjds/s13688-025-00588-0.

[8] A. Durairaj, A. Baskaran, R. Janakiraman, and P. Narayanan, “An Optimized Deep Learning-Based Smart Parking Mechanism for Smart City Environment,” International Journal of Computational Intelligence Systems, vol. 18, article 290, 2025. doi: 10.1007/s44196-025-01030-z.

[9] K. Vakrinou, E. G. Mantouka, and E. Vlahogianni, “Leveraging multisource data for off-street parking occupancy prediction: a twofold analysis,” International Journal of Transportation Science and Technology, vol. 21, pp. 352–371, 2026. doi: 10.1016/j.ijtst.2025.02.005.

[10] A. Ostadi, D. Das, G. Elmamooz, and D. Nicklas, “Data Engineering for Horizon-Aware Smart Parking Systems,” Datenbank-Spektrum, vol. 26, no. 2, pp. 155–164, 2026. doi: 10.1007/s13222-026-00544-w.

[11] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[12] K. Cho et al., “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation,” in Proceedings of EMNLP, 2014, pp. 1724–1734. doi: 10.3115/v1/D14-1179.

[13] A. Vaswani et al., “Attention Is All You Need,” in Advances in Neural Information Processing Systems 30, 2017. arXiv:1706.03762.

[14] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in Proceedings of IEEE International Conference on Neural Networks, vol. 4, 1995, pp. 1942–1948. doi: 10.1109/ICNN.1995.488968.

[15] M. Dorigo, V. Maniezzo, and A. Colorni, “Ant System: Optimization by a Colony of Cooperating Agents,” IEEE Transactions on Systems, Man, and Cybernetics, Part B, vol. 26, no. 1, pp. 29–41, 1996. doi: 10.1109/3477.484436.

[16] NXP Semiconductors, “MFRC522 Standard performance MIFARE and NTAG frontend,” Rev. 3.9, 2016. https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf (accessed 26 Oct. 2026).

[17] TowerPro, “SG90 Digital,” manufacturer specifications. https://towerpro.com.tw/product/sg90-7/ (accessed 26 Oct. 2026).

[18] Espressif Systems, “ESP32 Series Datasheet,” v5.3. https://documentation.espressif.com/esp32_datasheet_en.html (accessed 26 Oct. 2026).

Dataset Availability

The privacy-safe simulated dataset archive and project source are publicly available from GitHub. The version-pinned ZIP contains parking_raw.csv, parking_features.csv, dataset_metadata.json, and dataset documentation. The project Drive folder includes the project materials and demonstration video; access follows the folder's sharing permissions.

GitHub project repository

Public dataset ZIP (GitHub)

Project folder and demonstration video (Google Drive)

Appendix A. Current ESP32 Arduino controller source

This appendix reproduces the checked-in controller implementation and its public configuration template. The example configuration contains no real Wi-Fi credentials or device token.

firmware/esp32_controller/src/main.cpp

```
#include <Arduino.h>
#include <Adafruit_SSD1306.h>
#include <ArduinoJson.h>
#include <ESP32Servo.h>
#include <HTTPClient.h>
#include <MFRC522.h>
#include <Preferences.h>
#include <SPI.h>
#include <WiFi.h>

#include "pins.h"

#if __has_include("local_config.h")
#include "local_config.h"
#else
static constexpr char WIFI_SSID[] = "";
static constexpr char WIFI_PASSWORD[] = "";
static constexpr char BACKEND_BASE_URL[] = "http://192.168.1.10:8000";
static constexpr int SERVO_CLOSED_DEGREES = 10;
static constexpr int SERVO_OPEN_DEGREES = 85;
static constexpr uint32_t GATE_OPEN_MILLISECONDS = 2500;
static constexpr uint32_t STATUS_POLL_MILLISECONDS = 10000;
#endif

namespace {
constexpr uint32_t kHttpTimeoutMs = 3500;
constexpr uint32_t kWifiRetryMs = 5000;
constexpr uint32_t kCardRemovalMs = 500;
constexpr uint32_t kRetryInitialMs = 1000;
constexpr uint32_t kRetryMaximumMs = 30000;

Adafruit_SSD1306 display(128, 64, &Wire, -1);
MFRC522 rfid(Pins::RfidSs, Pins::RfidReset);
Servo gateServo;
Preferences preferences;

struct SlotState {
  bool occupied[5] = {false, false, false, false, false};
  bool known = false;
  String names[5];
};

SlotState slotState;
bool displayReady = false;
bool storageReady = false;
bool gateReady = false;
bool wifiStarted = false;
bool wifiWasConnected = false;
uint32_t wifiDisconnectedAtMs = 0;
bool cardArmed = true;
bool absenceTiming = false;
uint32_t absentSinceMs = 0;
uint32_t lastWifiAttemptMs = 0;
uint32_t lastStatusMs = 0;
uint32_t nextRetryMs = 0;
uint32_t retryDelayMs = kRetryInitialMs;
bool gateOpen = false;
uint32_t gateCloseAtMs = 0;
bool pendingOperation = false;
bool pendingGateDone = false;
uint8_t telemetryMask = 0;
uint8_t telemetryAck = 0;
uint32_t nextTelemetryMs = 0;
String pendingUid;
String pendingKey;
String pendingType;
String assignedSlotName;
int assignedSlotId = 0;
String lastMessage = "Starting";
uint32_t scanReceivedAtMs = 0;

struct HttpResult {
  int code = 0;
  String body;
```

```
};

void emitMetric(const char *event, const char *operation = nullptr, bool hasSuccess = false,
                bool success = false, uint32_t latencyMs = 0, int httpStatus = 0) {
  JsonDocument metric;
  metric["event"] = event;
  metric["uptime_ms"] = millis();
  if (operation) metric["operation"] = operation;
  if (hasSuccess) metric["success"] = success;
  if (latencyMs) metric["latency_ms"] = latencyMs;
  if (httpStatus) metric["http_status"] = httpStatus;
  metric["free_heap_bytes"] = ESP.getFreeHeap();
  serializeJson(metric, Serial);
  Serial.println();
}

void showMessage(const String &title, const String &line1 = "", const String &line2 = "") {
  lastMessage = title;
  Serial.println(title);  // Never log RFID values, request bodies, or credentials.
  if (!displayReady) return;
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("SMART PARKING");
  display.setTextSize(1);
  display.println(title);
  if (line1.length()) display.println(line1);
  if (line2.length()) display.println(line2);
  display.display();
}

String normalizedUid(const MFRC522::Uid &uid) {
  String value;
  value.reserve(uid.size * 2);
  char byteText[3];
  for (byte index = 0; index < uid.size; ++index) {
    snprintf(byteText, sizeof(byteText), "%02X", uid.uidByte[index]);
    value += byteText;
  }
  return value;
}

String uppercase(String value) {
  value.toUpperCase();
  return value;
}

String backendUrl(const char *path) {
  String base(BACKEND_BASE_URL);
  while (base.endsWith("/")) base.remove(base.length() - 1);
  return base + path;
}

byte transferRfidByteBitBanged(byte value) {
  byte received = 0;
  for (uint8_t bit = 0; bit < 8; ++bit) {
    digitalWrite(Pins::RfidMosi, (value & 0x80) ? HIGH : LOW);
    delayMicroseconds(2);
    digitalWrite(Pins::RfidSck, HIGH);
    received = static_cast<byte>((received << 1) |
                                 (digitalRead(Pins::RfidMiso) == HIGH ? 1 : 0));
    delayMicroseconds(2);
    digitalWrite(Pins::RfidSck, LOW);
    value <<= 1;
  }
  return received;
}

byte readRfidVersionBitBanged() {
  pinMode(Pins::RfidSs, OUTPUT);
  pinMode(Pins::RfidSck, OUTPUT);
  pinMode(Pins::RfidMosi, OUTPUT);
  pinMode(Pins::RfidMiso, INPUT_PULLUP);
  pinMode(Pins::RfidReset, OUTPUT);
  digitalWrite(Pins::RfidSs, HIGH);
  digitalWrite(Pins::RfidSck, LOW);
  digitalWrite(Pins::RfidReset, LOW);
  delayMicroseconds(10);
  digitalWrite(Pins::RfidReset, HIGH);
  delay(50);
  digitalWrite(Pins::RfidSs, LOW);
  delayMicroseconds(2);
```

```
  transferRfidByteBitBanged(0x80 | MFRC522::VersionReg);
  const byte version = transferRfidByteBitBanged(0x00);
  digitalWrite(Pins::RfidSs, HIGH);
  return version;
}

byte readRfidVersionAtLowClock(uint8_t chipSelectPin) {
  pinMode(chipSelectPin, OUTPUT);
  digitalWrite(Pins::RfidSs, HIGH);
  digitalWrite(chipSelectPin, HIGH);
  SPI.beginTransaction(SPISettings(250000, MSBFIRST, SPI_MODE0));
  digitalWrite(chipSelectPin, LOW);
  SPI.transfer(0x80 | MFRC522::VersionReg);
  const byte version = SPI.transfer(0x00);
  digitalWrite(chipSelectPin, HIGH);
  SPI.endTransaction();
  return version;
}

HttpResult requestHttp(const String &url, const String &method, const String &body = "",
                       uint32_t timeoutMs = kHttpTimeoutMs) {
  HttpResult result;
  HTTPClient http;
  WiFiClient network;
  http.setTimeout(timeoutMs);
  if (!http.begin(network, url)) return result;
  http.addHeader("Accept", "application/json");
  if (DEVICE_API_TOKEN[0] != '\0')
    http.addHeader("Authorization", String("Bearer ") + DEVICE_API_TOKEN);
  if (method == "POST") http.addHeader("Content-Type", "application/json");
  result.code = method == "POST" ? http.POST(body) : http.GET();
  if (result.code > 0) {
    const int contentLength = http.getSize();
    if (contentLength > 0 && contentLength <= 16384) result.body = http.getString();
  }
  http.end();
  return result;
}

bool checkHealth() {
  HttpResult response = requestHttp(backendUrl("/health"), "GET");
  if (response.code != HTTP_CODE_OK) return false;
  JsonDocument document;
  if (deserializeJson(document, response.body) != DeserializationError::Ok) return false;
  return document["status"].is<const char *>() &&
         String(document["status"].as<const char *>()) == "ok";
}

void setUnknownLeds() {
  slotState.known = false;
}

void runLedSelfTest() {
  for (size_t index = 0; index < Pins::SlotCount; ++index) {
    const uint8_t pin = Pins::SlotLeds[index];
    digitalWrite(pin, HIGH);
    Serial.printf("{\"event\":\"led_self_test\",\"slot\":%u,\"gpio\":%u}\n",
                  static_cast<unsigned>(index + 1), static_cast<unsigned>(pin));
    delay(250);
    digitalWrite(pin, LOW);
    delay(100);
  }
}

void refreshLeds() {
  static bool previousKnown = false;
  static bool previousOutputs[Pins::SlotCount] = {};
  const bool blink = ((millis() / 400) % 2) == 0;
  for (size_t index = 0; index < Pins::SlotCount; ++index) {
    const bool on = slotState.known ? slotState.occupied[index] : blink;
    digitalWrite(Pins::SlotLeds[index], on ? HIGH : LOW);
    if (slotState.known && (!previousKnown || previousOutputs[index] != on)) {
      Serial.printf(
          "{\"event\":\"slot_led\",\"slot\":%u,\"gpio\":%u,\"occupied\":%s,\"commanded_on\":%s}\n",
          static_cast<unsigned>(index + 1), static_cast<unsigned>(Pins::SlotLeds[index]),
          slotState.occupied[index] ? "true" : "false", on ? "true" : "false");
    }
    previousOutputs[index] = on;
  }
  previousKnown = slotState.known;
}

bool parseStatus(const String &body, SlotState &result) {
```

```
  JsonDocument document;
  if (deserializeJson(document, body) != DeserializationError::Ok) return false;
  JsonArray slots = document["slots"].as<JsonArray>();
  if (slots.isNull() || slots.size() != Pins::SlotCount) return false;
  bool seen[5] = {false, false, false, false, false};
  for (JsonObject slot : slots) {
    const int id = slot["id"] | 0;
    if (id < 1 || id > static_cast<int>(Pins::SlotCount) || seen[id - 1] ||
        !slot["occupied"].is<bool>() || !slot["name"].is<const char *>()) return false;
    seen[id - 1] = true;
    result.occupied[id - 1] = slot["occupied"].as<bool>();
    result.names[id - 1] = slot["name"].as<String>();
  }
  result.known = true;
  return true;
}

bool fetchStatus(SlotState &result) {
  if (WiFi.status() != WL_CONNECTED) return false;
  HttpResult response = requestHttp(backendUrl("/api/parking/device-status"), "GET");
  if (response.code != HTTP_CODE_OK || !parseStatus(response.body, result)) return false;
  return true;
}

bool activeSessionForUid(const String &body, const String &uid, int *slotId = nullptr) {
  JsonDocument document;
  if (deserializeJson(document, body) != DeserializationError::Ok) return false;
  JsonObject vehicle = document["vehicle"].as<JsonObject>();
  if (!vehicle.isNull() && vehicle["rfid_uid"].is<const char *>() && uppercase(vehicle["rfid_uid"].as<String>()) == uid) {
    if (slotId) *slotId = vehicle["id"] | 0;
    return true;
  }
  JsonArray slots = document["slots"].as<JsonArray>();
  if (slots.isNull()) return false;
  for (JsonObject slot : slots) {
    if (!slot["rfid_uid"].is<const char *>()) continue;
    if (uppercase(slot["rfid_uid"].as<String>()) == uid) {
      if (slotId) *slotId = slot["id"] | 0;
      return true;
    }
  }
  return false;
}

bool statusForUid(const String &uid, bool &parked) {
  HttpResult response = requestHttp(backendUrl((String("/api/parking/device-status?rfid_uid=") + uid).c_str()), "GET");
  if (response.code != HTTP_CODE_OK) return false;
  SlotState parsed;
  if (!parseStatus(response.body, parsed)) return false;
  parked = activeSessionForUid(response.body, uid);
  slotState = parsed;
  refreshLeds();
  return true;
}

bool persistPending() {
  if (!storageReady || preferences.putString("uid", pendingUid) == 0 ||
      preferences.putString("key", pendingKey) == 0 ||
      preferences.putString("type", pendingType) == 0 ||
      preferences.putBool("gate_done", pendingGateDone) != 1 ||
      preferences.putUChar("telemetry", telemetryMask) != 1 ||
      preferences.putUChar("telemetry_ack", telemetryAck) != 1 ||
      preferences.putBool("pending", true) != 1) return false;
  pendingOperation = true;
  return true;
}

void clearPending() {
  if (!storageReady) return;
  preferences.putBool("pending", false);
  preferences.remove("uid");
  preferences.remove("key");
  preferences.remove("type");
  preferences.putBool("gate_done", false);
  preferences.putUChar("telemetry", 0);
  preferences.putUChar("telemetry_ack", 0);
  pendingUid = "";
  pendingKey = "";
  pendingType = "";
  pendingOperation = false;
  pendingGateDone = false;
  telemetryMask = 0;
  telemetryAck = 0;
```

```
  assignedSlotId = 0;
  assignedSlotName = "";
  retryDelayMs = kRetryInitialMs;
}

String newIdempotencyKey() {
  const uint32_t sequence = preferences.getUInt("seq", 0) + 1;
  if (!storageReady || preferences.putUInt("seq", sequence) != sizeof(sequence)) return "";
  char key[64];
  snprintf(key, sizeof(key), "esp32-%012llx-%lu",
           static_cast<unsigned long long>(ESP.getEfuseMac()),
           static_cast<unsigned long>(sequence));
  return String(key);
}

bool beginOperation(const String &uid, const String &operation) {
  if (!storageReady) return false;
  pendingUid = uid;
  pendingKey = newIdempotencyKey();
  if (!pendingKey.length()) return false;
  pendingType = operation;
  scanReceivedAtMs = millis();
  pendingGateDone = false;
  telemetryMask = 1;
  telemetryAck = 0;
  if (!persistPending()) {
    pendingUid = "";
    pendingKey = "";
    pendingType = "";
    return false;
  }  // Persist before POST, so retries and reboots reuse this exact key.
  nextRetryMs = millis();
  return true;
}

bool verifyEntryCommit(const String &uid, int expectedSlotId) {
  HttpResult status = requestHttp(backendUrl((String("/api/parking/device-status?rfid_uid=") + uid).c_str()), "GET");
  if (status.code != HTTP_CODE_OK) return false;
  int actualSlotId = 0;
  if (!activeSessionForUid(status.body, uid, &actualSlotId) || actualSlotId != expectedSlotId)
    return false;
  SlotState parsed;
  if (!parseStatus(status.body, parsed)) return false;
  slotState = parsed;
  refreshLeds();
  return true;
}

bool verifyExitCommit(const String &uid) {
  HttpResult status = requestHttp(backendUrl((String("/api/parking/device-status?rfid_uid=") + uid).c_str()), "GET");
  if (status.code != HTTP_CODE_OK) return false;
  if (activeSessionForUid(status.body, uid)) return false;
  SlotState parsed;
  if (!parseStatus(status.body, parsed)) return false;
  slotState = parsed;
  refreshLeds();
  return true;
}

bool openGateOnce() {
  if (pendingGateDone) return true;
  if (!storageReady || !gateReady || preferences.putBool("gate_done", true) != 1) return false;
  // Mark before moving: a reset can fail safe closed, but cannot replay the actuation.
  pendingGateDone = true;
  gateServo.write(SERVO_OPEN_DEGREES);
  gateOpen = true;
  gateCloseAtMs = millis() + GATE_OPEN_MILLISECONDS;
  emitMetric("gate_command", pendingType.c_str(), false, false,
             millis() - scanReceivedAtMs);
  telemetryMask |= 2;
  preferences.putUChar("telemetry", telemetryMask);
  return true;
}

void flushTelemetry() {
  if (!pendingOperation || WiFi.status() != WL_CONNECTED ||
      static_cast<int32_t>(millis() - nextTelemetryMs) < 0) return;
  for (uint8_t phase = 1; phase <= 4; phase <<= 1) {
    if (!(telemetryMask & phase) || (telemetryAck & phase)) continue;
    JsonDocument payload;
    payload["event_type"] = phase == 1 ? "rfid_read" : phase == 2 ? "gate_opened" : "gate_closed";
    payload["operation"] = pendingType;
    payload["operation_key"] = pendingKey;
```

```
    payload["rfid_uid"] = pendingUid;
    String encoded;
    serializeJson(payload, encoded);
    const HttpResult result = requestHttp(backendUrl("/api/parking/hardware-events"), "POST", encoded,
                                          gateOpen ? 350 : kHttpTimeoutMs);
    if (result.code != HTTP_CODE_OK && result.code != HTTP_CODE_CREATED) {
      nextTelemetryMs = millis() + 1000;
      return;
    }
    telemetryAck |= phase;
    preferences.putUChar("telemetry_ack", telemetryAck);
  }
  if (pendingGateDone && !gateOpen && (telemetryMask & 4) &&
      (telemetryAck & telemetryMask) == telemetryMask) clearPending();
}

void processPendingOperation() {
  if (!pendingOperation || gateOpen || !storageReady || WiFi.status() != WL_CONNECTED ||
      static_cast<int32_t>(millis() - nextRetryMs) < 0) return;
  if (pendingGateDone || !(telemetryAck & 1)) return;

  JsonDocument request;
  request["rfid_uid"] = pendingUid;
  request["idempotency_key"] = pendingKey;
  request["source"] = "controller";
  String encoded;
  serializeJson(request, encoded);
  const char *path = pendingType == "entry" ? "/api/vehicle/entry" : "/api/vehicle/exit";
  const uint32_t requestStartedMs = millis();
  HttpResult response = requestHttp(backendUrl(path), "POST", encoded);
  emitMetric("api_transaction", pendingType.c_str(), true,
             response.code == HTTP_CODE_OK || response.code == HTTP_CODE_CREATED,
             millis() - requestStartedMs, response.code);

  if (response.code == HTTP_CODE_OK || response.code == HTTP_CODE_CREATED) {
    JsonDocument result;
    if (deserializeJson(result, response.body) != DeserializationError::Ok ||
        !result["rfid_uid"].is<const char *>() ||
        uppercase(result["rfid_uid"].as<String>()) != pendingUid ||
        String(result["gate_action"] | "") != "open") {
      showMessage("Invalid server reply", "Gate remains closed", "Retrying safely");
      nextRetryMs = millis() + retryDelayMs;
      retryDelayMs = min(retryDelayMs * 2, kRetryMaximumMs);
      return;
    }

    bool committed = false;
    if (pendingType == "entry") {
      const int slotId = result["slot_id"] | 0;
      const String slotName = result["slot_name"] | "";
      if (slotId >= 1 && slotName.length() &&
          verifyEntryCommit(pendingUid, slotId)) {
        assignedSlotId = slotId;
        assignedSlotName = slotName;
        committed = true;
        showMessage("Entry successful", "Assigned " + slotName);
      }
    } else if (pendingType == "exit" && verifyExitCommit(pendingUid)) {
      committed = true;
      showMessage("Exit successful", "Session closed");
    }
    if (!committed) {
      showMessage("Commit unverified", "Gate remains closed", "Retrying same request");
      nextRetryMs = millis() + retryDelayMs;
      retryDelayMs = min(retryDelayMs * 2, kRetryMaximumMs);
      return;
    }

    if (!pendingGateDone && !openGateOnce()) {
      showMessage("Storage/gate error", "Gate remains closed", "Retrying safely");
      nextRetryMs = millis() + retryDelayMs;
      retryDelayMs = min(retryDelayMs * 2, kRetryMaximumMs);
    }
    return;
  }

  if (response.code >= 400 && response.code < 500 && response.code != HTTP_CODE_REQUEST_TIMEOUT) {
    if (response.code == 409 && pendingType == "entry")
      showMessage("Parking full", "No slot assigned", "Gate remains closed");
    else showMessage("Request rejected", "HTTP " + String(response.code), "Gate remains closed");
    clearPending();
    return;
  }
```

```
  showMessage("Network/server error", "HTTP " + String(response.code), "Retrying same key");
  nextRetryMs = millis() + retryDelayMs;
  retryDelayMs = min(retryDelayMs * 2, kRetryMaximumMs);
}

bool cardStillOnReader() {
  byte atqa[2];
  byte atqaSize = sizeof(atqa);
  const MFRC522::StatusCode status = rfid.PICC_WakeupA(atqa, &atqaSize);
  if (status == MFRC522::STATUS_OK || status == MFRC522::STATUS_COLLISION) {
    rfid.PICC_HaltA();
    return true;
  }
  return false;
}

bool cardPresentForScan() {
  if (rfid.PICC_IsNewCardPresent()) return true;
  byte atqa[2];
  byte atqaSize = sizeof(atqa);
  const MFRC522::StatusCode status = rfid.PICC_WakeupA(atqa, &atqaSize);
  return status == MFRC522::STATUS_OK || status == MFRC522::STATUS_COLLISION;
}

void pollCard() {
  if (!cardArmed) {
    if (cardStillOnReader()) {
      absenceTiming = false;
    } else if (!absenceTiming) {
      absenceTiming = true;
      absentSinceMs = millis();
    } else if (millis() - absentSinceMs >= kCardRemovalMs) {
      cardArmed = true;
      absenceTiming = false;
      if (!pendingOperation) showMessage("Ready", "Scan RFID card");
    }
    return;
  }

  if (!cardPresentForScan() || !rfid.PICC_ReadCardSerial()) return;
  scanReceivedAtMs = millis();
  cardArmed = false;
  absenceTiming = false;
  const String uid = normalizedUid(rfid.uid);
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
  if (pendingOperation) {
    showMessage("Request pending", "Wait for server", "Then remove card");
    return;
  }
  if (rfid.uid.size != 4 && rfid.uid.size != 7 && rfid.uid.size != 10) {
    showMessage("Malformed RFID", "Card UID length invalid");
    return;
  }
  emitMetric("rfid_read", nullptr, true, true);
  if (WiFi.status() != WL_CONNECTED) {
    setUnknownLeds();
    showMessage("Wi-Fi unavailable", "Gate remains closed");
    return;
  }
  showMessage("RFID detected", "Checking session...");
  bool parked = false;
  if (!statusForUid(uid, parked)) {
    setUnknownLeds();
    showMessage("Server unavailable", "Gate remains closed");
    return;
  }
  if (!beginOperation(uid, parked ? "exit" : "entry")) {
    showMessage("Storage error", "Request not sent", "Gate remains closed");
    return;
  }
  if (parked) showMessage("Exit request", "Confirming session", "Gate remains closed");
  else showMessage("Entry request", "Allocating a slot", "Gate remains closed");
}

void updateWifi() {
  const bool connected = WiFi.status() == WL_CONNECTED;
  if (connected && !wifiWasConnected) {
    wifiWasConnected = true;
    emitMetric("wifi_connected", nullptr, false, false,
               wifiDisconnectedAtMs ? millis() - wifiDisconnectedAtMs : 0);
    wifiDisconnectedAtMs = 0;
```

```
  } else if (!connected && wifiWasConnected) {
    wifiWasConnected = false;
    wifiDisconnectedAtMs = millis();
    emitMetric("wifi_disconnected");
  }
  if (WIFI_SSID[0] == '\0' || connected) return;
  setUnknownLeds();
  if (!wifiStarted) {
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    wifiStarted = true;
    showMessage("Connecting Wi-Fi...");
    lastWifiAttemptMs = millis();
  } else if (millis() - lastWifiAttemptMs >= kWifiRetryMs) {
    WiFi.reconnect();
    lastWifiAttemptMs = millis();
    showMessage("Reconnecting Wi-Fi");
  }
}

void updateGate() {
  if (gateOpen && static_cast<int32_t>(millis() - gateCloseAtMs) >= 0) {
    gateServo.write(SERVO_CLOSED_DEGREES);
    emitMetric("gate_command", pendingType.c_str(), false, false,
               millis() - scanReceivedAtMs);
    gateOpen = false;
    if (pendingOperation) {
      telemetryMask |= 4;
      preferences.putUChar("telemetry", telemetryMask);
    }
    showMessage("Gate closed", "Ready for next vehicle");
  }
}
}  // namespace

void setup() {
  Serial.begin(115200);
  for (uint8_t pin : Pins::SlotLeds) {
    digitalWrite(pin, LOW);
    pinMode(pin, OUTPUT);
  }
  runLedSelfTest();
  refreshLeds();  // Blinking means availability is unknown until a status fetch succeeds.

  Wire.begin(Pins::I2cSda, Pins::I2cScl);
  displayReady = display.begin(SSD1306_SWITCHCAPVCC, Pins::OledAddress);
  showMessage("Initializing RFID");

  const byte bitBangVersion = readRfidVersionBitBanged();
  SPI.begin(Pins::RfidSck, Pins::RfidMiso, Pins::RfidMosi, Pins::RfidSs);
  rfid.PCD_Init();
  const byte readerVersion = rfid.PCD_ReadRegister(MFRC522::VersionReg);
  pinMode(Pins::RfidSs, OUTPUT);
  digitalWrite(Pins::RfidSs, HIGH);
  const byte lowClockVersion = readRfidVersionAtLowClock(Pins::RfidSs);
  pinMode(Pins::RfidMiso, INPUT_PULLUP);
  const bool misoIdleHigh = digitalRead(Pins::RfidMiso) == HIGH;
  const bool resetPinHigh = digitalRead(Pins::RfidReset) == HIGH;
  JsonDocument readerMetric;
  readerMetric["event"] = "rfid_reader";
  readerMetric["uptime_ms"] = millis();
  readerMetric["version_register"] = readerVersion;
  readerMetric["bit_banged_version_register"] = bitBangVersion;
  readerMetric["low_clock_version_register"] = lowClockVersion;
  readerMetric["reset_pin_high"] = resetPinHigh;
  readerMetric["miso_idle_high"] = misoIdleHigh;
  readerMetric["ready"] = readerVersion != 0x00 && readerVersion != 0xFF;
  readerMetric["free_heap_bytes"] = ESP.getFreeHeap();
  serializeJson(readerMetric, Serial);
  Serial.println();
  gateServo.setPeriodHertz(50);
  gateReady = gateServo.attach(Pins::GateServo, 500, 2400) >= 0;
  if (gateReady) gateServo.write(SERVO_CLOSED_DEGREES);
  else showMessage("Gate unavailable", "Gate remains closed");

  storageReady = preferences.begin("parking", false);
  emitMetric("controller_boot");
  if (!storageReady) showMessage("Storage unavailable", "RFID requests disabled");
  pendingOperation = storageReady && preferences.getBool("pending", false);
  if (pendingOperation) {
    pendingUid = preferences.getString("uid", "");
    pendingKey = preferences.getString("key", "");
    pendingType = preferences.getString("type", "");
```

```
    scanReceivedAtMs = millis();
    pendingGateDone = preferences.getBool("gate_done", false);
    telemetryMask = preferences.getUChar("telemetry", 0);
    telemetryAck = preferences.getUChar("telemetry_ack", 0);
    if (pendingGateDone) {
      telemetryMask |= 4;
      preferences.putUChar("telemetry", telemetryMask);
    }
    if (!pendingUid.length() || !pendingKey.length() ||
        (pendingType != "entry" && pendingType != "exit")) clearPending();
    else showMessage("Recovering request", "Using saved idempotency key");
  } else if (WIFI_SSID[0] == '\0') {
    showMessage("Wi-Fi not configured", "Copy local_config", "and restart");
  } else {
    showMessage("System initialized", "Connecting Wi-Fi...");
  }
  updateWifi();
}

void loop() {
  updateWifi();
  updateGate();
  refreshLeds();

  if (WiFi.status() == WL_CONNECTED) {
    flushTelemetry();
    if (millis() - lastStatusMs >= STATUS_POLL_MILLISECONDS) {
      lastStatusMs = millis();
      SlotState latest;
      if (checkHealth() && fetchStatus(latest)) {
        slotState = latest;
        refreshLeds();
        if (!pendingOperation && !gateOpen) showMessage("Ready", "Scan RFID card");
      } else {
        setUnknownLeds();
        showMessage("Backend status error", "Slot state unknown");
      }
    }
    processPendingOperation();
    if (!pendingOperation && !gateOpen) pollCard();
  }
  delay(5);
}
```

firmware/esp32_controller/include/pins.h

```
#pragma once

#include <Arduino.h>

namespace Pins {
constexpr uint8_t RfidSck = 18;
constexpr uint8_t RfidMiso = 19;
constexpr uint8_t RfidMosi = 23;
constexpr uint8_t RfidSs = 5;
constexpr uint8_t RfidReset = 4;

constexpr uint8_t I2cSda = 21;
constexpr uint8_t I2cScl = 22;
constexpr uint8_t OledAddress = 0x3C;

constexpr uint8_t SlotLeds[] = {25, 26, 27, 32, 33};
constexpr size_t SlotCount = sizeof(SlotLeds) / sizeof(SlotLeds[0]);
constexpr uint8_t GateServo = 14;
}  // namespace Pins
```

firmware/esp32_controller/include/local_config.example.h

```
#pragma once

// Copy to local_config.h and set values on the trusted local network.
// Never expose the backend control API to the public internet.
static constexpr char WIFI_SSID[] = "";
static constexpr char WIFI_PASSWORD[] = "";
static constexpr char BACKEND_BASE_URL[] = "http://192.168.1.10:8000";
static constexpr char DEVICE_API_TOKEN[] = "";

static constexpr int SERVO_CLOSED_DEGREES = 10;
static constexpr int SERVO_OPEN_DEGREES = 85;
```

```
static constexpr uint32_t GATE_OPEN_MILLISECONDS = 2500;
static constexpr uint32_t STATUS_POLL_MILLISECONDS = 10000;
```

firmware/esp32_controller/platformio.ini

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
build_flags = -DCORE_DEBUG_LEVEL=0
lib_deps =
    miguelbalboa/MFRC522@^1.4.12
    bblanchon/ArduinoJson@^7.2.1
    adafruit/Adafruit SSD1306@^2.5.13
    adafruit/Adafruit GFX Library@^1.11.11
    madhephaestus/ESP32Servo@^3.0.6
```